

UTILIZING MACHINE LEARNING TO ENHANCE EARLY INTERVENTION IN MENTAL HEALTH CARE

Submitted in partial fulfillment of the requirements for the

Award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

Submitted By

NURUKURTHY L V V ADITYA

20A31A1249

under the Esteemed Guidance of

Dr.M.UMA DEVI M.tech.,Ph.D.,

Associate Professor & HoD, Department of IT



DEPARTMENT OF INFORMATION TECHNOLOGY

**PRAGATI ENGINEERING COLLEGE
(AUTONOMOUS)**

(Approved by AICTE & Permanently Affiliated to JNTUK & Accredited by NBA and NAAC)
1-378, ADB Road, Surampalem, E.G.Dist., A.P, Pin-533437

2020-2024

PRAGATI ENGINEERING COLLEGE (AUTONOMOUS)

(Approved by AICTE & Permanently Affiliated to JNTUK & Accredited by NBA and NAAC)

1-378, ADB Road, Surampalem, E.G.Dist., A.P, Pin-533437.



CERTIFICATE

DEPARTMENT OF INFORMATION TECHNOLOGY

*This is to certify that the report entitled “Utilizing machine learning to enhance early intervention in mental health care”, that is being submitted by **Nurukurthy L V V Aditya(20A31A1249)**, in partial fulfillment for the award of the Degree of Bachelor of Technology, during the year **2020-2024** in Information Technology, Pragati Engineering College is a record of bonafide work carried out by them.*

Head of the Department

Dr.M.Uma Devi M.Tech.,Phd.,

Associate Professor & HoD,Department of IT

Project Supervisor

Dr.M.Uma Devi M.Tech.,Phd.,

Associate Professor & HoD,Department of IT

External Examiner

ACKNOWLEDGEMENT

It gives us an immense pleasure to express a deep sense of gratitude to our guide **Dr.M.Uma Devi**, Associate Professor and Head of the Department of Information Technology because of her wholehearted and valuable guidance throughout the report. Without her sustained and sincere effort, this report would not have taken this shape. She encouraged and helped us to overcome various difficulties that we have faced at various stages of our report.

We would like to sincerely thank **Dr.M.Uma Devi**, Associate Professor and Head of the Department of Information Technology, for providing all the necessary facilities that led to the successful completion of our report.

We would like to take this opportunity to thank our beloved Principal and Vice-principal, **Dr.K.SATYANARAYANA,Professor & Principal** for providing great support to us in completing our project and for giving us the opportunity of doing the project report.

We wish to express our special thanks to our beloved **Dr.G.NARESH ,Professor of EEE& Vice Principal** for giving guidelines and encouragement.

We wish to express sincere gratitude to our beloved and respected **Dr. P. KRISHNA RAO,Chairman** and **Sri.M.V.HARANATHA BABU, Director (Management)**and **Sri. M. SATISH, Vice-President** for their encouragement and blessings.

We would like to thank all the faculty members of the Department of Computer Science Engineering for their direct or indirect support for helping us in completion of this report.

Nurukurthy L V V Aditya (20A31A1249)

ABSTRACT

Nowadays, people are becoming more and more concerned with their physical health, but mental health is not given the same level of attention. Even if they are aware that they have been afflicted by chronic mental illnesses, many people choose not to seek treatment out of fear of what others would think, a belief that they have lost their minds, a dislike of doctors, or all three. These circumstances make it urgently necessary to find a solution so that more individuals are not inclined to mental diseases. This paper focuses on forecasting mental health using deep learning approaches and machine learning algorithm that is support vector machine. Support vector machine is used to solve the existing problem, as many machine learning and deep learning techniques are helping to solve these contemporary difficulties. SVM gives more accuracy compared to other machine learning algorithms to predict the mental illness..

TABLE OF CONTENTS

ACKNOWLEDGMENT	iii
ABSTRACT	iv
LIST OF FIGURES	v
Chapter 1. Introduction	1
1.1 Objective of the project	2
Chapter 2. Literature Survey	3
Chapter 3. System Analysis	7
3.1 Existing System	7
3.2 Proposed System	8
Chapter 4. Essential Diagrams	9
4.1 System Architecture	9
4.2 UML Diagrams	10
Chapter 5. System Implementation	14
5.1 Modules& System Requirements	14
Chapter 6. System Testing	17
6.1 Types of Testing	17
6.2 System Study	21
Chapter 7. Experiments & Results	26
Chapter 8. Conclusion & Future Work	32
Reference	33
APPENDIX-A Source Code	35
PAPER PUBLICATION	

Figure No.	Name of the Figure	Page. No.
1	System Architecture Diagram	9
2	Class Diagram	10
3	Use Case Diagram	11
4	Sequential Diagram	12
5	Deployment Diagram	12
6	Activity Diagram	13
7	Hardware Requirements	15
8	Software Requirements	16
9	Software Testing types	17
10	Home Page	26
11	User login page	26
12	Registration Page	27
13	User OTP	27
14	Admin Login	28
15	Admin's dashboard	28
16	Pending users	29
17	Dataset upload	29
18	Comparison graph	30
19	Prediction form	30
20	Not Detected Graph	31
21	Detected Graph	31

CHAPTER-1

INTRODUCTION

A person's emotions, intelligence, and capacity for interpersonal communication are unquestionably all impacted by mental illness, which is a health problem. These issues have shown that mental illness has significant societal effects and calls for fresh preventative and therapeutic approaches. Implementing these strategies requires first performing early mental health identification. Mental disease diagnoses are frequently made based on the patient's self-report, which necessitates the use of questionnaires designed to pinpoint specific emotional or social interaction patterns. With the proper treatment and care, many persons with mental illnesses or emotional disorders should be able to recover. Artificial intelligence has a branch called machine learning that is widely employed nowadays.

It is thought to be a very helpful tool for predicting mental health. It is making it possible for several researchers to gather crucial information from the data, offer tailored experiences, and create automated intelligent systems. Future occurrences have been predicted and categorised using widely used machine learning methods including support vector machines, random forests, and artificial neural networks. In addition to giving clinicians a platform to analyse a vast amount of patient data and develop individualised treatments based on the patient's medical condition, machine learning (ML) is getting more adept at diagnosing diseases. As a result of the advancement of new technology, people have grown accustomed to working from home, and numerous software and IT companies have embraced this concept. Due of this, people rarely connect with one another and many do not live a healthy life at home. As a result, many people today struggle with despair, anxiety, and melancholy.

In the modern era, people place more emphasis on their physical health than on their mental health. Even when they are conscious of the fact they have long-term mental illnesses, many people choose not to seek treatment out of fear of being judged, the belief that they are insane, or simply because they detest doctors. The major goal of this method is to design a system that can accurately and faithfully preserve original work while also detecting users' mental health in advance.

1.1 OBJECTIVE OF THE PROJECT

1.1.1 Major Objectives:

Develop a Predictive Model: To create an accurate and reliable machine learning model that can predict mental health illnesses based on input features such as age, gender, family history, workplace environment, and health-related factors.
Improve Early Detection: To facilitate the early identification of mental health issues, enabling timely intervention and support for individuals at risk, thereby contributing to better health outcomes.
Enhance Accessibility to Mental Health Resources: To provide a tool that can be widely used by healthcare professionals and individuals, overcoming geographical and financial barriers to mental health services.

1.1.2 Minor Objectives:

Optimize Algorithm Performance: To compare and optimize various supervised learning algorithms, such as KNN, Logistic Regression, Decision Trees, and Boosting algorithms, to find the most effective model for this application.
Implement a User-Friendly Web Application: To develop and deploy the machine learning model within a Django-based web application, ensuring it is accessible, easy to use, and secure for both administrators and end-users.
Ensure Data Security and Privacy: To implement robust data handling and user authentication protocols to protect sensitive information and comply with privacy regulations.
Provide Real-Time Data Analysis and Feedback: To enable the web application to offer real-time analysis and feedback on model performance, allowing users to upload new data and receive instant predictions.

Facilitate Continuous Learning and Improvement: To design the system in a way that allows for easy updates and improvements as new data becomes available or as advancements in machine learning algorithms occur.

CHAPTER-2

LITERATURE SURVEY

TITLE: "Predicting Mental Health Illness using Machine Learning Algorithms."

Mental health illness prediction is a critical area of research in healthcare, with machine learning techniques offering promising avenues for accurate and timely diagnosis. In this study, we explore the application of various machine learning algorithms, including XGBoost, AdaBoost, K-Nearest Neighbors (KNN), Decision Tree, Logistic Regression, Random Forest, and Gradient Boosting, for predicting mental health illnesses.

XG Boost Algorithm:

XGBoost, or Extreme Gradient Boosting, is a cutting-edge machine learning algorithm that has gained widespread acclaim for its exceptional performance in various predictive modeling tasks, making it an invaluable tool in the data scientist's arsenal. Born out of the need for a scalable and efficient boosting algorithm, XGBoost has proven its mettle in competitions and real-world applications alike.

At its core, XGBoost is a boosting ensemble algorithm, meaning it combines multiple weak learners, typically decision trees, to create a robust and accurate predictive model. What sets XGBoost apart from traditional boosting methods is its innovative approach to gradient boosting, which optimizes a specified objective function through the iterative addition of new trees.

One of XGBoost's key strengths lies in its ability to handle complex datasets with ease, thanks to its implementation of regularization techniques such as tree pruning and column subsampling. These techniques help prevent overfitting and enhance the model's generalization capabilities, ensuring reliable performance on unseen data.

ADA Boost Algorithm:

AdaBoost, short for Adaptive Boosting, is a popular ensemble learning algorithm that is widely used for classification tasks. It belongs to the family of boosting algorithms, which combine multiple weak learners to create a strong learner. AdaBoost is particularly renowned for its ability to improve the performance of weak classifiers, making it a powerful tool in machine learning.

The key idea behind AdaBoost is to iteratively train a sequence of weak learners, where each learner focuses on the instances that the previous ones misclassified. In each iteration, AdaBoost assigns higher weights to the misclassified instances, effectively giving them more importance in the subsequent training rounds. This adaptive weighting scheme allows AdaBoost to progressively focus on the most challenging instances, leading to a more accurate final prediction.

During training, AdaBoost assigns weights to each training instance, initially setting them equally. After each iteration, the weights of misclassified instances are increased, while the weights of correctly classified instances are decreased. This process ensures that subsequent weak learners pay more attention to the previously misclassified instances, thereby improving overall performance.

K-Nearest Neighbors Algorithm:

The k-Nearest Neighbors (KNN) algorithm is a simple yet powerful supervised machine learning algorithm used for both classification and regression tasks. It operates on the principle that data points with similar features tend to belong to the same class or have similar output values. KNN is a non-parametric and lazy learning algorithm, meaning it doesn't make any assumptions about the underlying data distribution and doesn't require explicit training before making predictions.

In KNN, the prediction for a new data point is made by considering the majority class (for classification) or the average value (for regression) among its k nearest neighbors in the feature space. The "k" in KNN represents the number of neighbors to consider, and it's a hyperparameter that needs to be specified by the user.

To find the nearest neighbors of a data point, KNN calculates the distance between that point and every other point in the dataset. The most commonly used distance metric is the Euclidean distance, although other metrics like Manhattan distance or Minkowski distance can also be used. Once the distances are calculated, KNN selects the k nearest neighbors based on these distances.

Decision Tree Algorithm:

A Decision Tree is a versatile and intuitive supervised machine

learning algorithm used for both classification and regression tasks. It is a tree-like structure where each internal node represents a "decision" based on the value of a feature, each branch represents the outcome of that decision, and each leaf node represents the final decision or the predicted outcome.

Decision Trees make predictions by recursively partitioning the feature space into smaller and smaller regions based on the feature values, aiming to maximize the homogeneity of the target variable within each partition. The process of creating a decision tree involves selecting the best feature to split the data at each node. This selection is typically based on criteria such as information gain (for classification tasks) or variance reduction (for regression tasks).

The simplicity and interpretability of decision trees make them attractive for understanding the underlying data relationships and decision-making process. Moreover, decision trees are robust to outliers and can handle both numerical and categorical data without the need for preprocessing.

Logistic Regression:

Logistic Regression is a fundamental supervised machine learning algorithm used for binary classification tasks, where the goal is to predict the probability of a binary outcome based on one or more predictor variables. Despite its name, logistic regression is a classification algorithm, not a regression algorithm.

The key idea behind logistic regression is to model the probability that a given instance belongs to a particular class using a logistic function, also known as the sigmoid function. The sigmoid function maps any real-valued input to a value between 0 and 1, representing the probability of the positive class.

During training, logistic regression learns to find the decision boundary that best separates the two classes in the feature space. This decision boundary is typically linear in the input space, although it can be nonlinear when polynomial or interaction terms are included in the model. Logistic regression can also be extended to handle multiclass classification tasks through techniques such as one-vs-rest (OvR) or multinomial logistic regression.

Random Forests Algorithm:

Random Forest is a versatile and powerful ensemble learning algorithm

used for both classification and regression tasks. It operates by constructing a multitude of decision trees during training and outputs the mode (for classification) or the average prediction (for regression) of the individual trees.

The key idea behind Random Forest is to introduce randomness in two key aspects: the sampling of training data and the selection of features at each split in the decision trees.

Bootstrap Sampling (Bagging): Random Forest employs a technique called bootstrap sampling, where multiple random subsets of the training data are created with replacement. Each subset is then used to train a separate decision tree. This sampling strategy introduces diversity among the trees, reducing the risk of overfitting and improving generalization performance.

Random Feature Selection: At each split in the decision trees, Random Forest considers only a random subset of features rather than all available features. This random feature selection further enhances the diversity among the trees and prevents individual trees from dominating the ensemble. By introducing randomness in feature selection, Random Forest decorrelates the trees and improves overall model performance.

Gradient Boosting Algorithm:

The key concept behind Gradient Boosting is to minimize a loss function (e.g., mean squared error for regression, cross-entropy loss for classification) by adding weak learners to the ensemble. In each iteration, a new tree is fitted to the residuals (the difference between the actual target values and the predictions made by the current ensemble). The tree is trained to predict these residuals, effectively reducing the errors made by the previous ensemble.

The term "gradient" in Gradient Boosting refers to the fact that it uses gradient descent optimization to minimize the loss function. It computes the gradient of the loss function with respect to the predictions of the current ensemble and then fits a new weak learner (tree) to the negative gradient, effectively moving in the direction that minimizes the loss.

Gradient Boosting is highly flexible and can be customized using various parameters, such as the learning rate (shrinkage parameter), the number of trees in the ensemble, and the maximum depth of each tree. It's important to tune these parameters carefully to prevent overfitting and achieve optimal performance.

CHAPTER-3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing system for diagnosing and managing mental health conditions primarily relies on traditional methods, including self-reporting by patients, clinical assessments by healthcare professionals, and standardized psychological tests. These approaches, while valuable, often face limitations related to subjectivity, accessibility, and scalability. Many individuals may not seek help due to stigma, lack of awareness, or insufficient resources, leading to underdiagnosis and undertreatment of mental health issues. Furthermore, the current system struggles to accommodate the high demand for mental health services, resulting in long waiting times and reduced effectiveness of interventions. The reliance on manual diagnosis and the absence of personalized, data-driven tools also hinder the ability to predict mental health conditions early and accurately, limiting preventive measures and timely support for those at risk.

LIMITATIONS OF EXISTING SYSTEM

Subjectivity and Variability in Diagnosis: Traditional diagnostic methods often rely heavily on self-reported symptoms and clinical judgments, which can introduce subjectivity and variability.

Access and Availability Issues: Many individuals face barriers to accessing mental health services due to geographic limitations, socioeconomic factors, and the stigma associated with mental health issues.

Scalability and Resource Constraints: The existing healthcare system often struggles with scalability, especially in the context of mental health services. There is a global shortage of mental health professionals relative to the demand for services, leading to long wait times for patients and overburdened practitioners.

Limited Predictive Capabilities: Current methods are more focused on diagnosing existing conditions rather than predicting potential mental health issues before they fully manifest. This reactive approach misses opportunities for early intervention that could prevent the development or worsening of mental health conditions.

AIM & OBJECTIVES

This study introduces the prediction of detecting the mental health illness using machine learning technologies.

3.2 PROPOSED SYSTEM

Our proposed system introduces a transformative approach to mental health diagnostics by integrating machine learning algorithms. It uses smart computer programs to guess if someone might have mental health problems. It looks at lots of information about a person and how healthy they are. This helps it make predictions about mental health issues before they become serious. The proposed system uses advanced machine learning techniques, with a primary focus on ADA Boost, to enhance the accuracy, efficiency, and effectiveness of diagnosing mental health conditions, potentially overcoming limitations of existing diagnostic frameworks.

ADVANTAGES OF PROPOSED SYSTEM

Objective and Data-Driven Diagnosis: The system relies on data and machine learning algorithms, reducing the subjectivity associated with traditional mental health diagnosis and providing objective assessments.

Improved Accuracy: Machine learning models, particularly Support Vector Machine, can provide more accurate and consistent predictions, reducing the likelihood of misdiagnosis and allowing for early intervention.

Scalability: The proposed system can scale to accommodate a large number of patients, addressing the current shortage of mental health professionals and making mental health services more widely accessible.

Stigma Reduction: By offering a non-judgmental and data-driven approach, the system can help reduce the stigma around mental health issues, encouraging individuals to seek help without fear of social repercussions.

Early Intervention: Early detection of mental health issues through the system's predictive capabilities allows for timely intervention and treatment, potentially preventing the worsening of conditions.

CHAPTER-4

ESSENTIAL DIAGRAMS

4.1 SYSTEM ARCHITECTURE

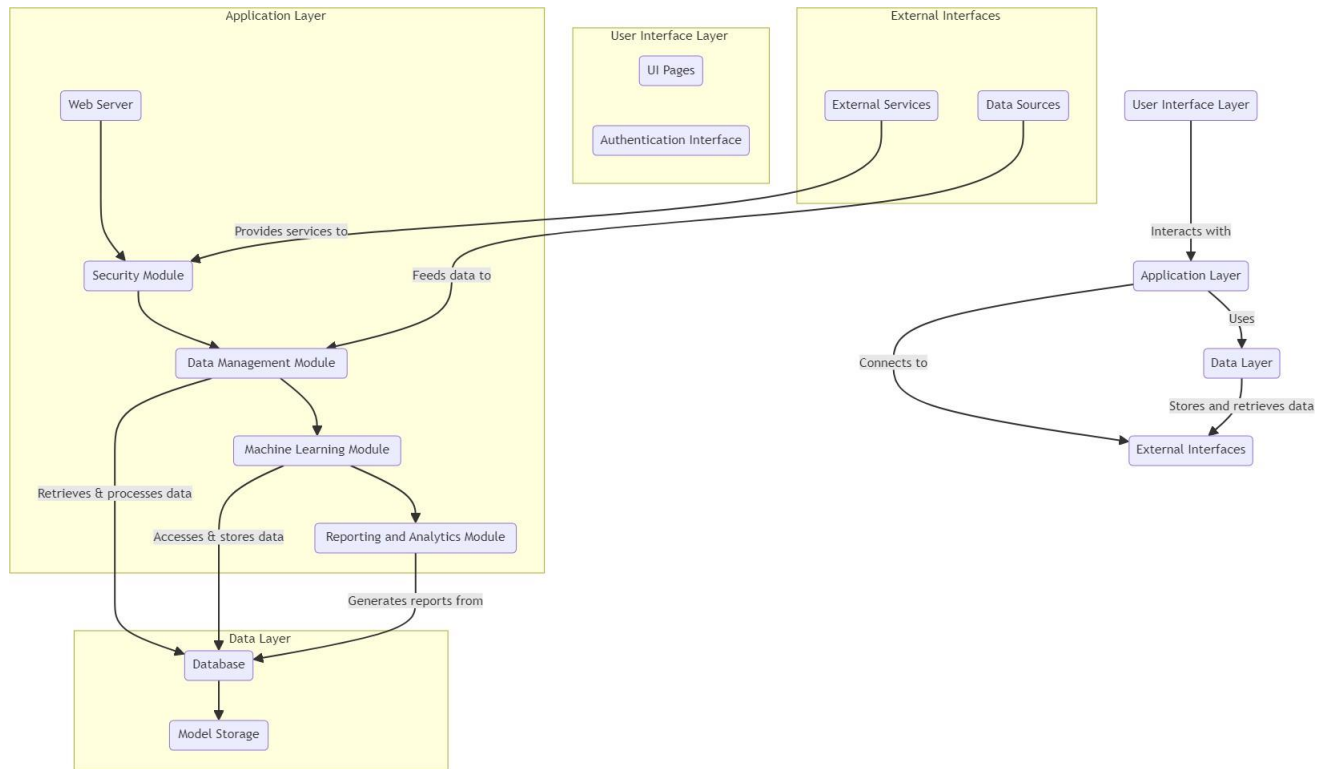


Fig.1 system architecture

- The UI Layer interacts directly with the Web Server, sending requests (like data uploads, prediction queries) and receiving responses (such as prediction results, analytics reports).
- The Web Server uses the Security Module for operations requiring authentication and authorization, ensuring secure processing of requests.
- The Data Management Module retrieves data from both the Database and external sources, processes it, and stores it back in the database for future use.

4.2 UML DIAGRAMS:

Class Diagram:

The class diagram is the main building block of object oriented modeling. It is used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. In the diagram, classes are represented with boxes which contain three parts:

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake

Class Diagram:

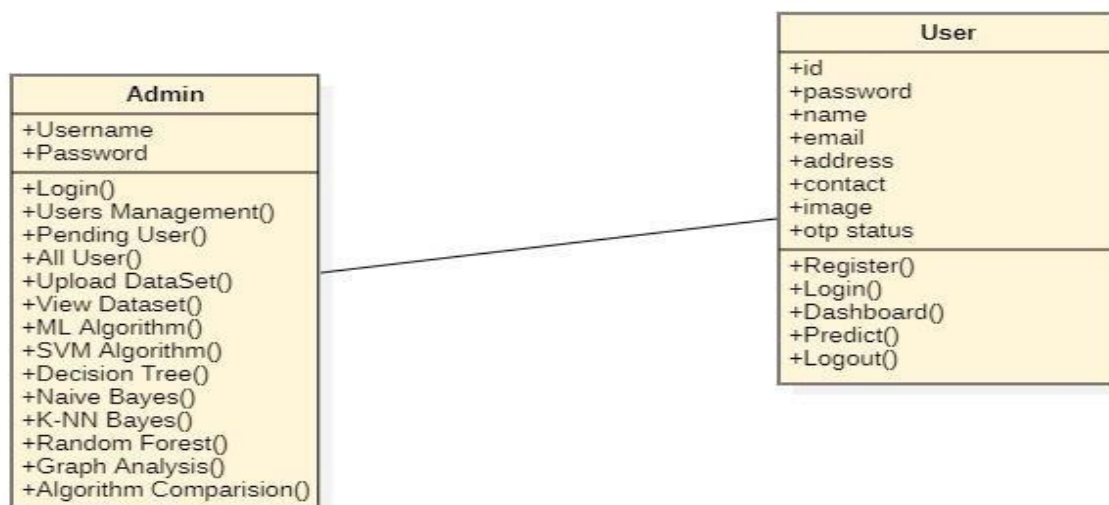


Fig.2 Class Diagram

Usecase Diagram:

A **usecase diagram** at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system.

This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

Use case Diagram:

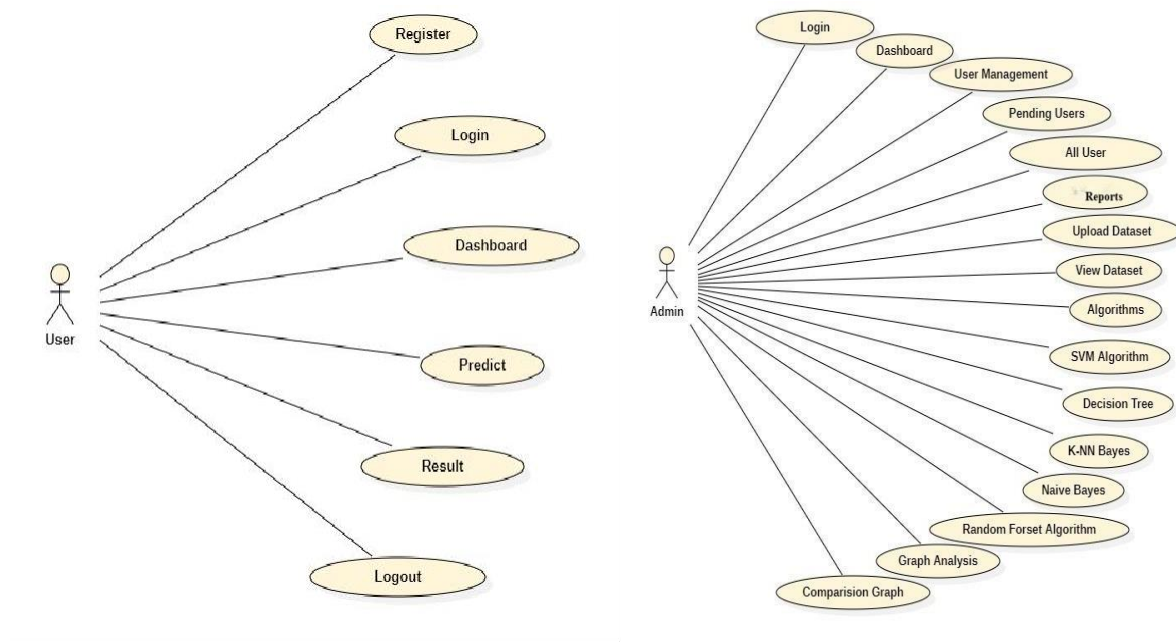


Fig.3 Use Case Diagram

Sequence diagram:

A **sequence diagram** is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called **event diagrams**, **event scenarios**, and timing diagrams.

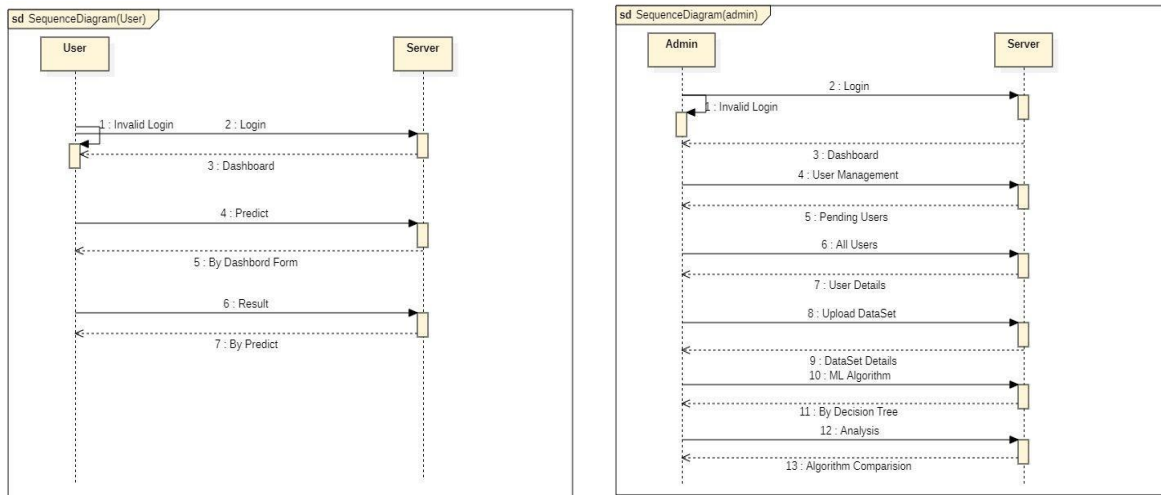


Fig.4 Sequence Diagram

Deployment Diagram:

A **deployment diagram** in the Unified Modeling Language models the *physical* deployment of artifacts on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components ("artifacts") run on each node (e.g., web application, database), and how the different pieces are connected (e.g. JDBC, REST, RMI).

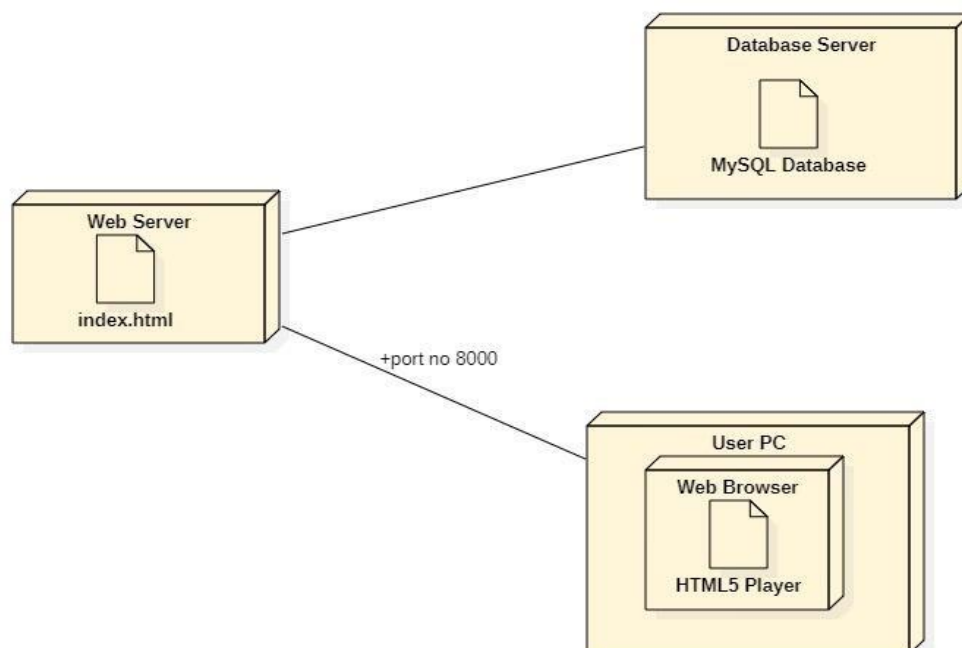


fig.5Deployment Diagram

Activity Diagram:

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. It is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. So the control flow is drawn from one operation to another. This flow can be sequential, branched or concurrent

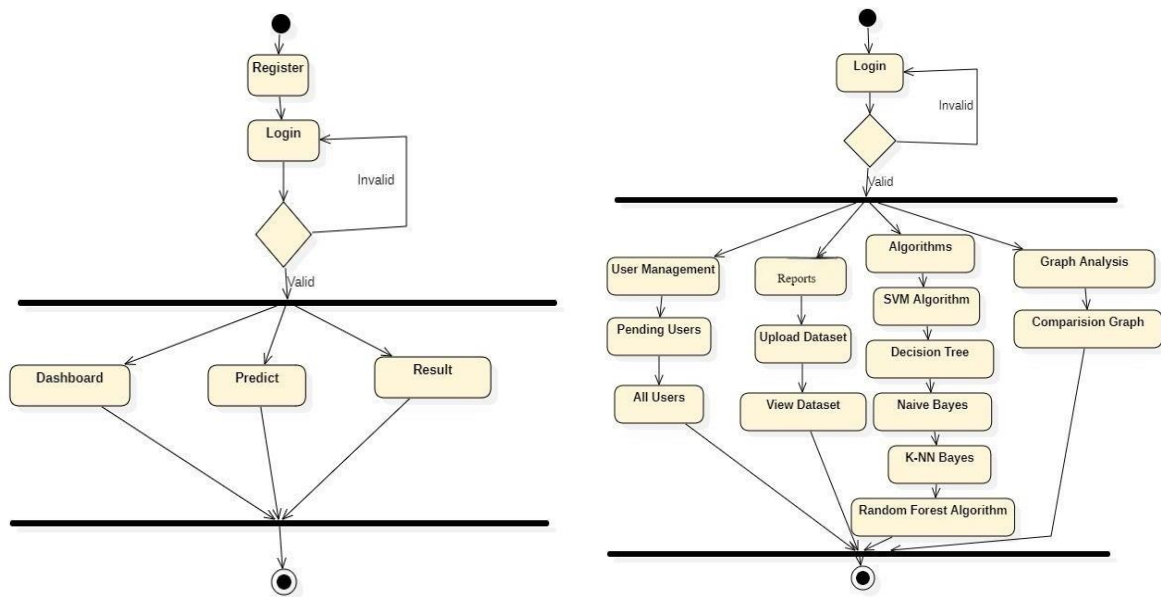


Fig 6. Activity Diagram

CHAPTER-5

SYSTEM IMPLEMENTATION

5.1 MODULES

1. Data Management Module

- **Data Collection:** Import data from various sources, including user inputs and external databases.
- **Data Preprocessing:** Clean, normalize, and transform data to make it suitable for analysis.
- **Data Storage:** Securely store processed data in a database for future retrieval and analysis.
- **Data Visualization:** Provide visual insights into the dataset through graphs and charts to help users understand data distributions and patterns.

2. Machine Learning Module

- **Model Training:** Train models using different algorithms (KNN, Logistic Regression, Decision Trees, etc.) on the preprocessed data.
- **Model Evaluation:** Assess the performance of each model using metrics such as accuracy, precision, recall, and F1 score.
- **Model Selection:** Choose the best-performing model(s) based on evaluation metrics for deployment.
- **Prediction:** Use the selected model to make predictions on new data and output the likelihood of mental health issues.

3. User Interface (UI) Module

- **User Dashboard:** A central dashboard for users to interact with the system, including data upload, initiating model training, and viewing predictions.
- **Admin Interface:** A dedicated interface for administrators to manage datasets, run models, and analyze system performance.

- **Authentication and Authorization:** Manage user login, registration, and access controls to ensure secure access to the system.

4. Security Module

- **Data Encryption:** Encrypt sensitive data both at rest and in transit to protect against unauthorized access.
- **User Authentication:** Implement secure authentication mechanisms, such as passwords, two-factor authentication, or OAuth.
- **Access Control:** Define and enforce user permissions to ensure users can only access data and functionalities relevant to their role.

5. Reporting and Analytics Module

- **Performance Reporting:** Generate reports on the performance of machine learning models, including metrics and comparative analyses.
- **Predictive Analytics:** Offer detailed insights and interpretations of prediction results to users, helping them understand the factors influencing the predictions.

Trend Analysis: Analyze and report on trends within the data over time, aiding in the identification of broader patterns related to mental health issues.

5.2 SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS

MINIMUM (Required for Execution)		SYSTEM used for Development	
System	Pentium IV 2.2 GHz	i3 Processor 5 th Gen	
Hard Disk	20GB	500GB	
RAM	1GB	4GB	

Fig.7 Hardware requirements

SOFTWARE REQUIREMENTS:

Operating System	Windows 10/11
Development Software	Python 3.10.8
Integrated Development Environment (IDE)	Visual Studio Code
Front End Technologies	HTML5, CSS3, Java Script
Web Development Framework	Django
RDBMS	MySQL Version
Application Database Software	WAMP or XAMPP Server
Web Server or Deployment Server	Django Application Development Server
Design/Modelling	Rational Rose

Fig 8. Software requirements

CHAPTER-6

SYSTEM TESTING

6.1 TYPES OF TESTING

We, as testers, are aware of the various types of Software Testing like Functional Testing, Non-Functional Testing, Automation Testing, Agile Testing, and their sub-types, etc. Each type of testing has its own features, advantages, and disadvantages as well. However, in this tutorial, we have covered mostly each and every type of software testing which we usually use in our day-to-day testing life.

Different Types of Software Testing

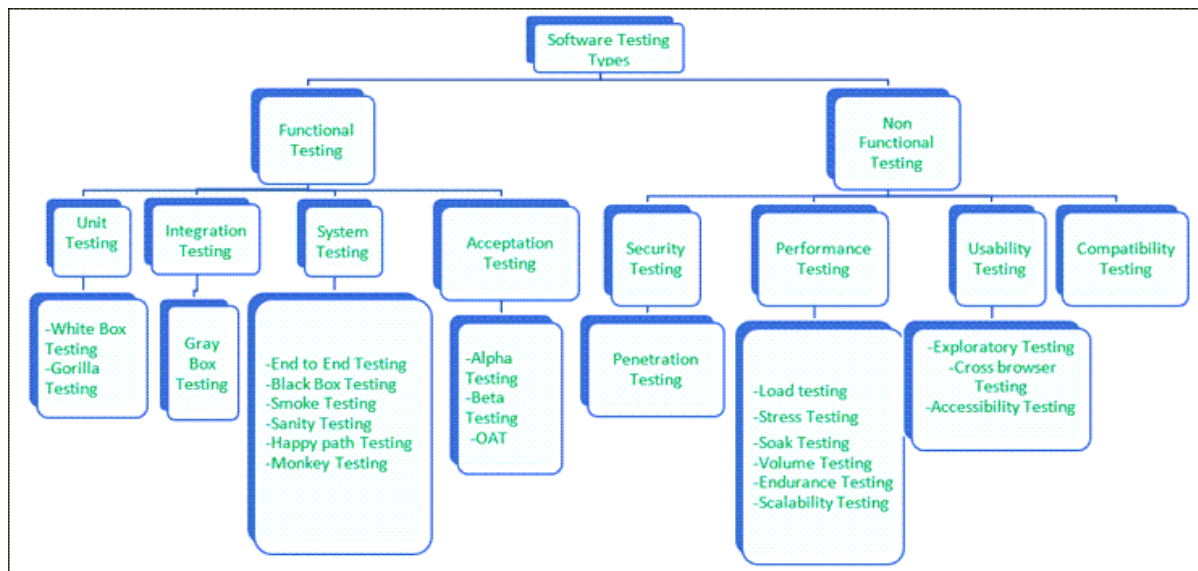


Fig 9. Software Testing

6.1.1 Functional Testing

There are four main types of functional testing.

#1) Unit Testing

Unit testing is a type of software testing which is done on an individual unit or component to test its corrections. Typically, Unit testing is done by the developer at the application development phase. Each unit in unit testing can be viewed as a method, function, procedure, or object. Developers often use test automation tools such as NUnit, Xunit, JUnit for the test execution.

a) White Box Testing

White box testing is a test technique in which the internal structure or code of an application is visible and accessible to the tester. In this technique, it is easy to find

loopholes in the design of an application or fault in business logic. Statement coverage and decision coverage/branch coverage are examples of white box test techniques.

b) Gorilla Testing

Gorilla testing is a test technique in which the tester and/or developer test the module of the application thoroughly in all aspects. Gorilla testing is done to check how robust your application is.

#2) Integration Testing

Integration testing is a type of software testing where two or more modules of an application are logically grouped together and tested as a whole. The focus of this type of testing is to find the defect on interface, communication, and data flow among modules. Top-down or Bottom-up approach is used while integrating modules into the whole system.

a) Gray box testing

As the name suggests, gray box testing is a combination of white-box testing and black-box testing. Testers have partial knowledge of the internal structure or code of an application.

#3) System Testing

System testing is types of testing where tester evaluates the whole system against the specified requirements.

a) End to End Testing

It involves testing a complete application environment in a situation that mimics real-world use, such as interacting with a database, using network communications, or interacting with other hardware, applications, or systems if appropriate.

b) Black Box Testing

Blackbox testing is a software testing technique in which testing is performed without knowing the internal structure, design, or code of a system under test. Testers should focus only on the input and output of test objects.

c) Smoke Testing

Smoke testing is performed to verify that basic and critical functionality of the system under test is working fine at a very high level. Whenever a new build is provided by the development team, then the Software Testing team validates the build and ensures that no major issue exists. The testing team will ensure that the build is stable, and a detailed level of testing will be carried out further.

d) Sanity Testing

Sanity testing is performed on a system to verify that newly added functionality or bug fixes are working fine. Sanity testing is done on stable build. It is a subset of the regression test.

e) Happy path Testing

The objective of Happy Path Testing is to test an application successfully on a positive flow. It does not look for negative or error conditions. The focus is only on valid and positive inputs through which the application generates the expected output.

f) Monkey Testing

Monkey Testing is carried out by a tester, assuming that if the monkey uses the application, then how random input and values will be entered by the Monkey without any knowledge or understanding of the application.

#4) Acceptance Testing

Acceptance testing is a type of testing where client/business/customer test the software with real time business scenarios.

The client accepts the software only when all the features and functionalities work as expected. This is the last phase of testing, after which the software goes into production. This is also called User Acceptance Testing (UAT).

6.1.2 Non-Functional Testing

There are four main types of functional testing.

#1) Security Testing

It is a type of testing performed by a special team. Any hacking method can penetrate the system.

Security Testing is done to check how the software, application, or website is secure from internal and/or external threats. This testing includes how much software is secure from malicious programs, viruses and how secure & strong the authorization and authentication processes.

a) Penetration Testing

Penetration Testing or Pen testing is the type of security testing performed as an authorized cyberattack on the system to find out the weak points of the system in terms of security. Pen testing is performed by outside contractors, generally known as ethical hackers. That is why it is also known as ethical hacking.

#2) Performance Testing

Performance testing is testing of an application's stability and response time by applying load. The word stability means the ability of the application to withstand in the presence of load. Response time is how quickly an application is available to users. Performance testing is done with the help of tools. Loader.IO, JMeter, LoadRunner, etc. are good tools available in the market.

a) Load testing

Load testing is testing of an application's stability and response time by applying load, which is equal to or less than the designed number of users for an application.

b) Stress Testing

Stress testing is testing an application's stability and response time by applying load, which is more than the designed number of users for an application.

c) Scalability Testing

Scalability testing is testing an application's stability and response time by applying load, which is more than the designed number of users for an application.

Let's say my application is giving response time as follows:

1000 users -2 sec

1400 users -2 sec

4000 users -3 sec

5000 users -45 sec

5150 users- crash – This is the point that needs to identify in scalability testing

d) Volume testing (flood testing)

Volume testing is testing an application's stability and response time by transferring a large volume of data to the database. Basically, it tests the capacity of the database to handle the data.

e) Endurance Testing (Soak Testing)

Endurance testing is testing an application's stability and response time by applying load continuously for a longer period to verify that the application is working fine.

#3) Usability Testing

Usability testing is testing an application from the user's perspective to check the look and feel and user-friendliness.

a) Exploratory testing

Exploratory Testing is informal testing performed by the testing team. The objective of this testing is to explore the application and look for defects that exist in the application. Testers

use the knowledge of the business domain to test the application. Test charters are used to guide the exploratory testing.

b) Cross browser testing

Cross browser testing is testing an application on different browsers, operating systems, mobile devices to see look and feel and performance. Why do we need cross-browser testing? The answer is different users use different operating systems, different browsers, and different mobile devices. The goal of the company is to get a good user experience regardless of those devices.

c) Accessibility Testing

The aim of Accessibility Testing is to determine whether the software or application is accessible for disabled people or not.

#4) Compatibility testing

This is a testing type in which it validates how software behaves and runs in a different environment, web servers, hardware, and network environment

6.2 SYSTEM STUDY

A feasibility study assesses the operational, technical and economic merits of the proposed project. The feasibility study is intended to be a preliminary review of the facts to see if it is worthy of proceeding to the analysis phase. From the systems analyst perspective, the feasibility analysis is the primary tool for recommending whether to proceed to the next phase or to discontinue the project.

The feasibility study is a management-oriented activity. The objective of a feasibility study is to find out if an information system project can be done and to suggest possible alternative solutions.

Projects are initiated for two broad reasons:

1. Problems that lend themselves to systems solutions
2. Opportunities for improving through:
 - (a) upgrading systems
 - (b) altering systems

(c) installing new systems

A feasibility study should provide management with enough information to decide:

- Whether the project can be done
- Whether the final product will benefit its intended users and organization
- What are the alternatives among which a solution will be chosen
- Is there a preferred alternative?

6.2.1 TECHNICAL FEASIBILITY

A large part of determining resources has to do with assessing technical feasibility. It considers the technical requirements of the proposed project. The technical requirements are then compared to the technical capability of the organization. The systems project is considered technically feasible if the internal technical capability is sufficient to support the project requirements.

The analyst must find out whether current technical resources can be upgraded or added to in a manner that fulfils the request under consideration.

The essential questions that help in testing the operational feasibility of a system include the following:

- Is the project feasible within the limits of current technology?
- Does the technology exist at all?
- Is it available within given resource constraints?
- Is it a practical proposition?
- Manpower- programmers, testers & debuggers
- Software and hardware
- Are the current technical resources sufficient for the new system?
- Can they be upgraded to provide to provide the level of technology necessary for the new system?
- Do we possess the necessary technical expertise, and is the schedule reasonable?
- Can the technology be easily applied to current problems?
- Does the technology have the capacity to handle the solution?
- Do we currently possess the necessary technology?

6.2.2 OPERATIONAL FEASIBILITY

Operational feasibility is dependent on human resources available for the project and involves projecting whether the system will be used if it is developed and implemented.

Operational feasibility is a measure of how well a proposed system solves the problems, and takes advantage of the opportunities identified during scope definition and how it satisfies the requirements identified in the requirements analysis phase of system development.

Operational feasibility reviews the willingness of the organization to support the proposed system. This is probably the most difficult of the feasibilities to gauge. In order to determine this feasibility, it is important to understand the management commitment to the proposed project.

The essential questions that help in testing the operational feasibility of a system include the following:

- Does current mode of operation provide adequate throughput and response time?
- Does current mode provide end users and managers with timely, pertinent, accurate and useful formatted information?
- Does current mode of operation provide cost-effective information services to the business?
- Could there be a reduction in cost and or an increase in benefits?
- Does current mode of operation offer effective controls to protect against fraud and to guarantee accuracy and security of data and information?
- Does current mode of operation make maximum use of available resources, including people, time, and flow of forms?
- Does current mode of operation provide reliable services
- Are the services flexible and expandable?
- Are the current work practices and procedures adequate to support the new system?
- If the system is developed, will it be used?
- Manpower problems
- Labour objections
- Manager resistance
- Organizational conflicts and policies
- Social acceptability

- Government regulations
- Does management support the project?
- Are the users not happy with current business practices?
- Will it reduce the time (operation) considerably?
- Have the users been involved in the planning and development of the project?
- Will the proposed system really benefit the organization?
- Does the overall response increase?
- Will accessibility of information be lost?
- Will the system affect the customers in considerable way?

6.2.3 ECONOMIC FEASIBILITY

Economic analysis could also be referred to as cost/benefit analysis. It is the most frequently used method for evaluating the effectiveness of a new system. In economic analysis the procedure is to determine the benefits and savings that are expected from a candidate system and compare them with costs. If benefits outweigh costs, then the decision is made to design and implement the system. An entrepreneur must accurately weigh the cost versus benefits before taking an action.

Possible questions raised in economic analysis are:

- Is the system cost effective?
- Do benefits outweigh costs?
- The cost of doing full system study
- The cost of business employee time
- Estimated cost of hardware
- Estimated cost of software/software development
- Is the project possible, given the resource constraints?
- What are the savings that will result from the system?
- Cost of employees' time for study
- Cost of packaged software/software development
- Selection among alternative financing arrangements (rent/lease/purchase)

The concerned business must be able to see the value of the investment it is pondering before committing to an entire system study. If short-term costs are not overshadowed by long-term gains or produce no immediate reduction in operating costs, then the system is not economically feasible, and the project should not proceed any further. If the expected benefits equal or exceed costs, the system can be judged to be economically feasible. Economic analysis is used for evaluating the effectiveness of the proposed system. The economic feasibility will review the expected costs to see if they are in-line with the projected budget or if the project has an acceptable return on investment. At this point, the projected costs will only be a rough estimate. The exact costs are not required to determine economic feasibility. It is only required to determine if it is feasible that the project costs will fall within the target budget or return on investment. A rough estimate of the project schedule is required to determine if it would be feasible to complete the systems project within a required timeframe. The required timeframe would need to be set by the organization

CHAPTER-7

EXPERIMENTS&RESULTS

STEP-1:Click on user in the home page.

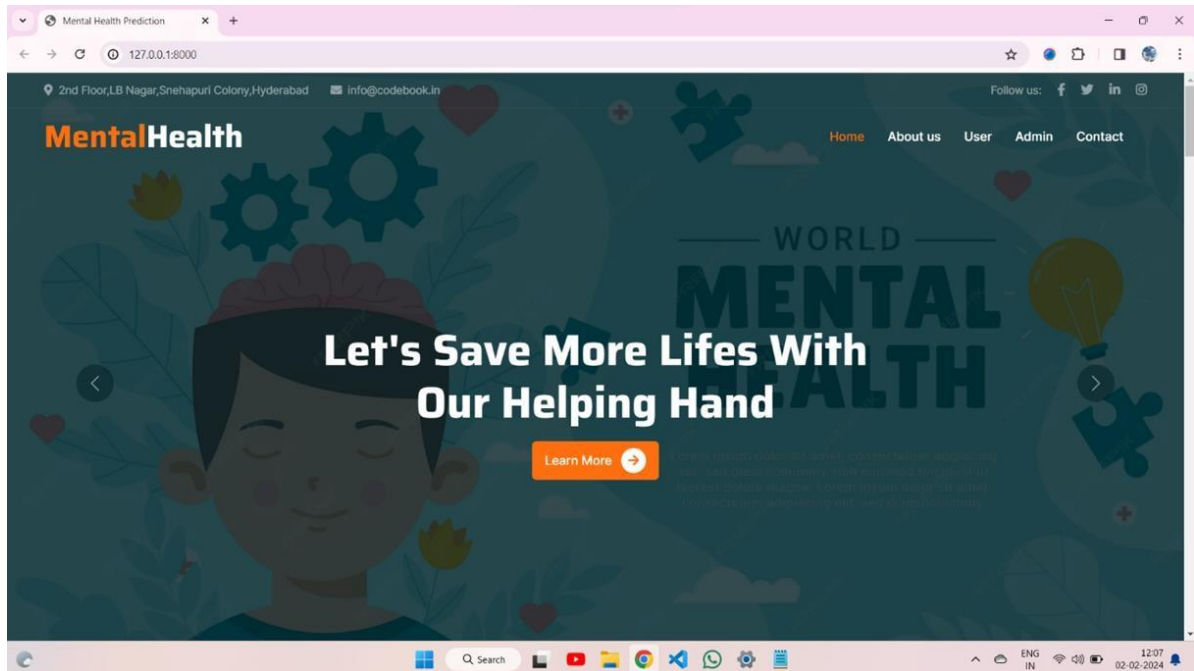


fig 10. Home page

STEP-2:Login with your registered mail and password.If not click register here.

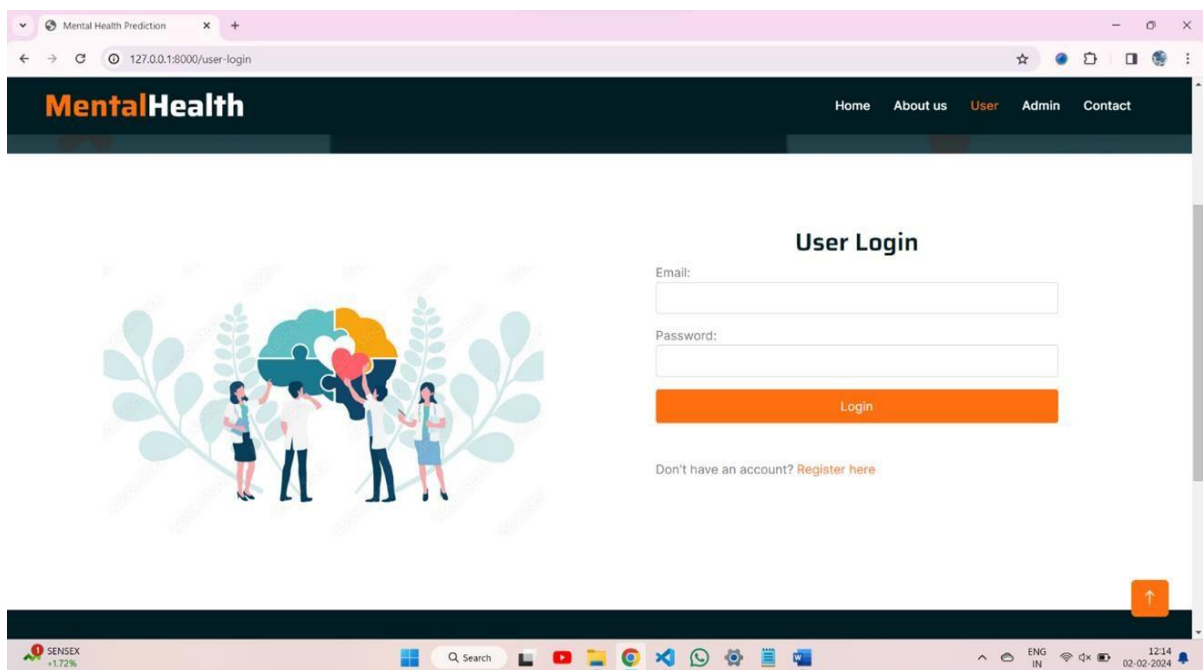
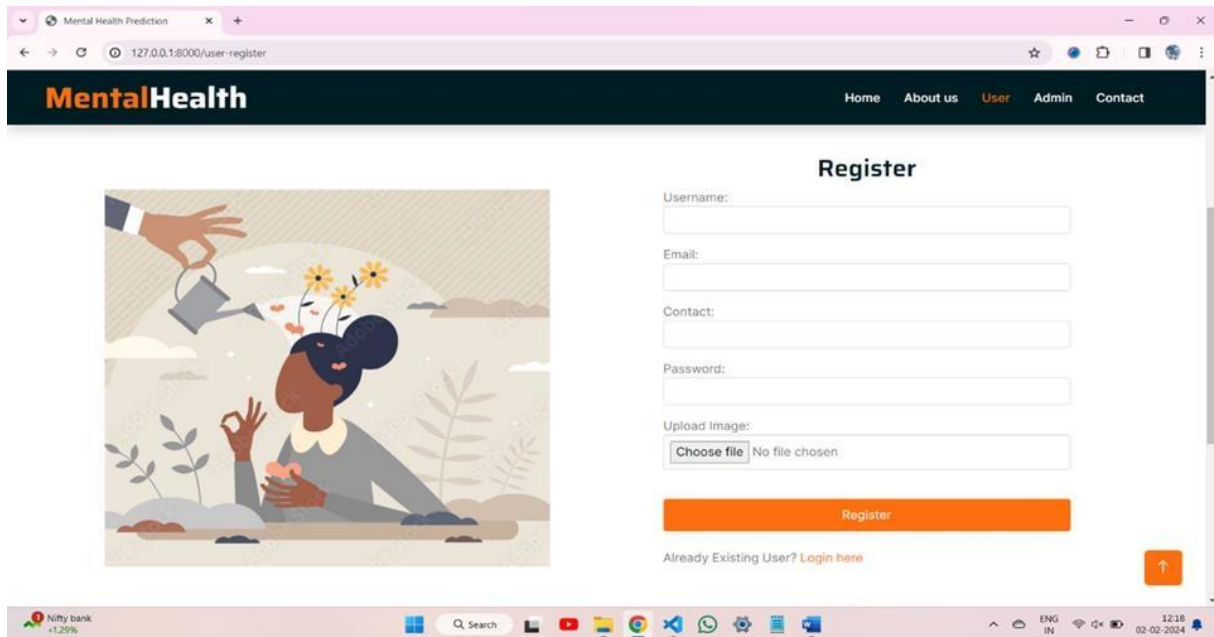


fig 11. User login Page

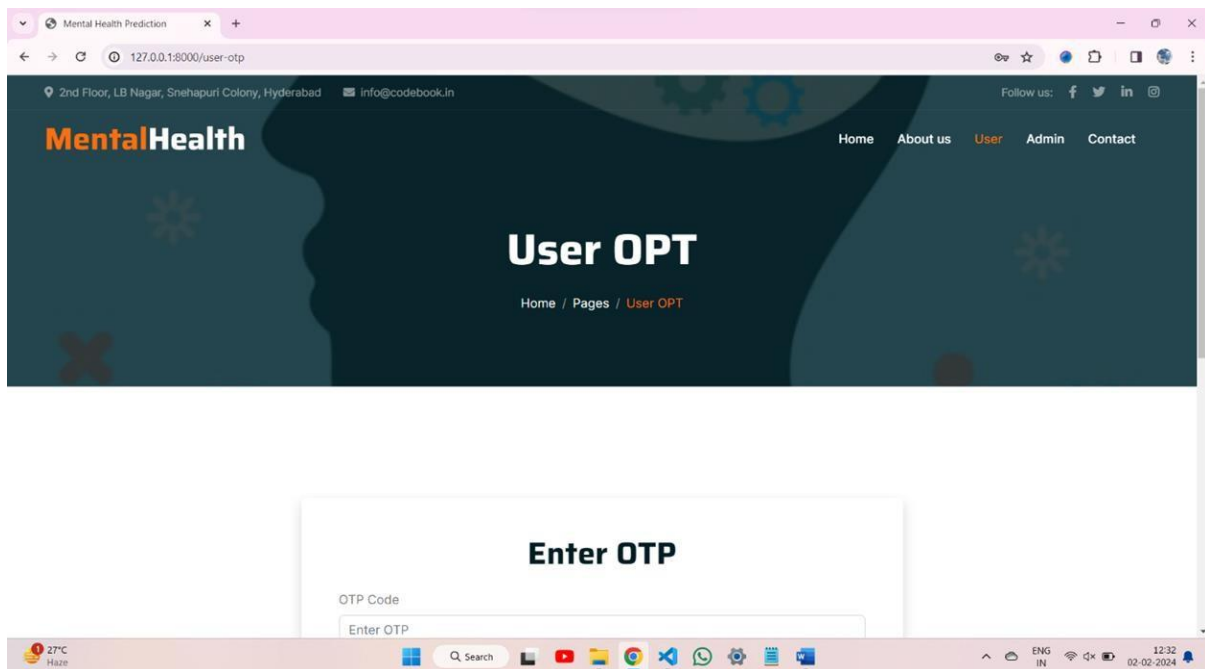
STEP-3:Give user details in register page and click on register.



The screenshot shows a web browser window with the URL `127.0.0.1:8000/user-register`. The page has a dark header with the "MentalHealth" logo and navigation links: Home, About us, User, Admin, and Contact. The main content area is titled "Register" and features a large illustration on the left of a person with flowers growing from their head, being watered by a hand. On the right, there is a registration form with the following fields: Username, Email, Contact, Password, and an Upload Image section with a "Choose file" button. Below the form is an orange "Register" button and a link for "Already Existing User? Login here". The browser's taskbar at the bottom shows various application icons and the system clock.

fig 12. Registration Page

STEP-4:User will get the OTP to their given mobile number.



The screenshot shows a web browser window with the URL `127.0.0.1:8000/user-otp`. The page has a dark header with the "MentalHealth" logo and navigation links: Home, About us, User, Admin, and Contact. The main content area is titled "User OPT" (likely a typo for OTP) and features a large illustration of a person's head in profile with gears inside. Below the illustration, there is a white box titled "Enter OTP" with a text input field labeled "Enter OTP". The browser's taskbar at the bottom shows various application icons and the system clock.

fig 13. User otp

STEP-5: Now the admin must login with their credentials and accept the pending users.

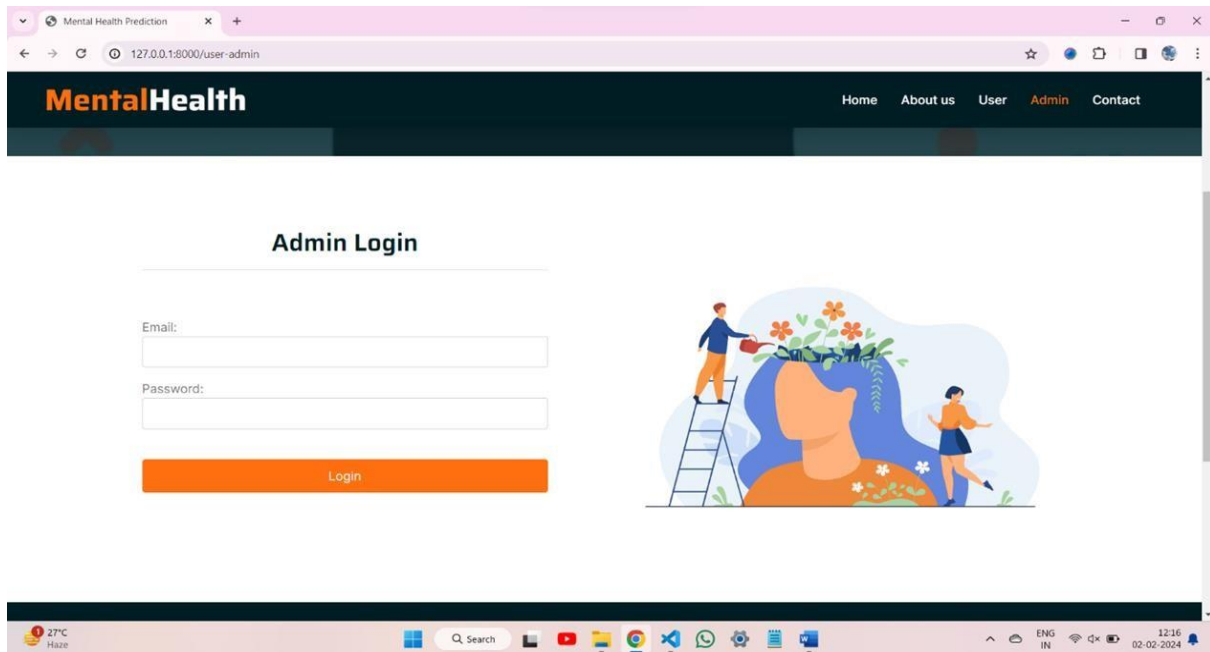


fig 14. Admin login

STEP-6: Now we can see the admin's dashboard.

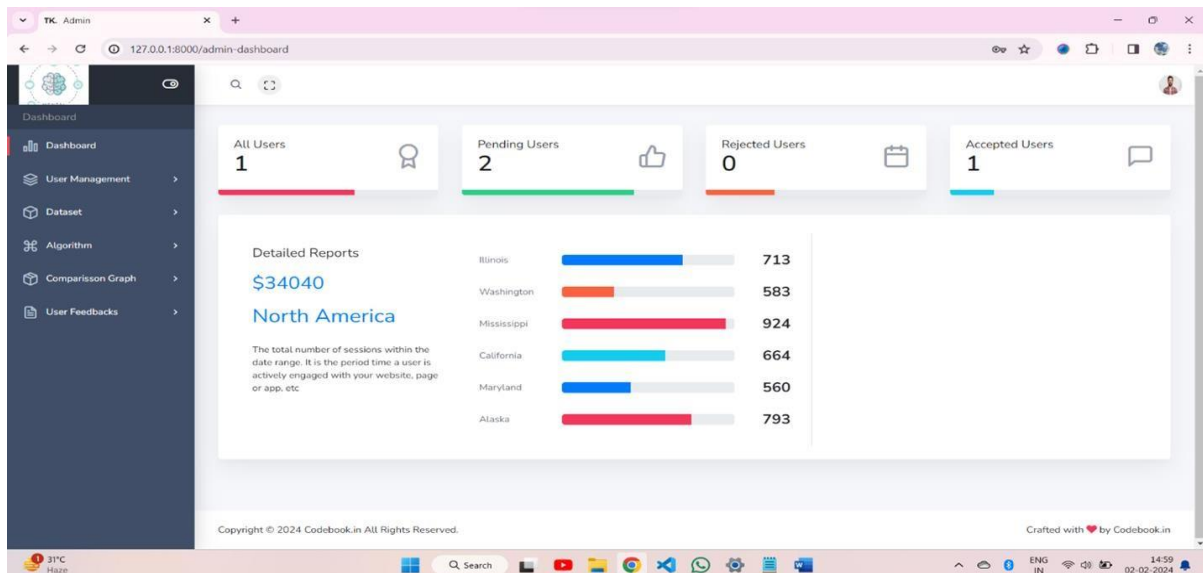


Fig 15. Admin's dashboard

STEP-7: Now the admin must accept the requested pending users.

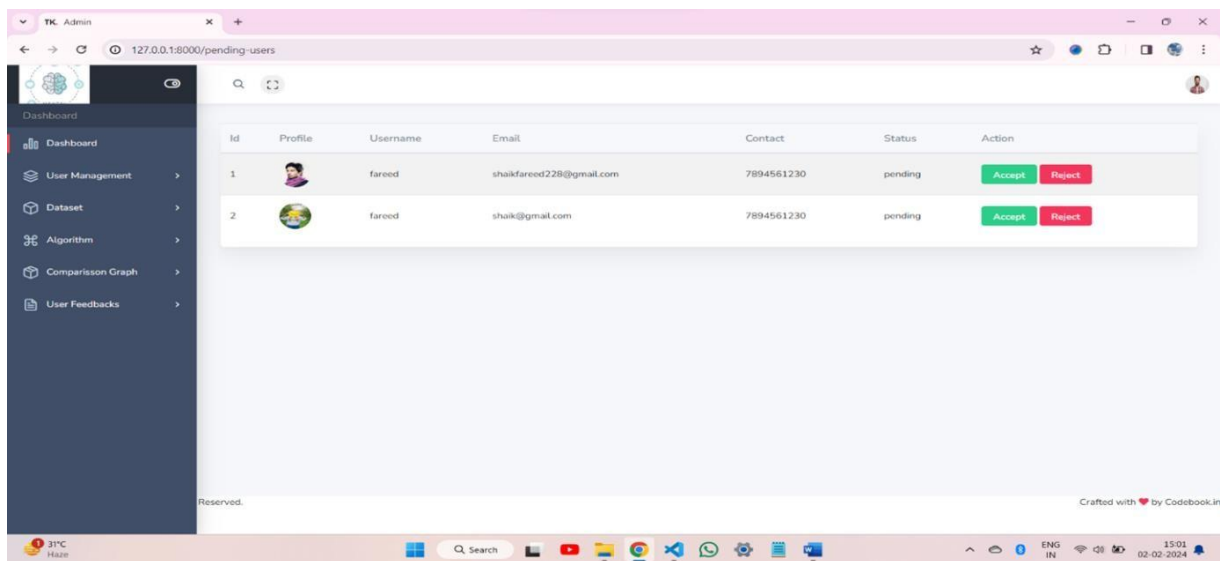


fig 16. Pending users

STEP-8: In the next step, the admin should upload the dataset and run with the given algorithms.

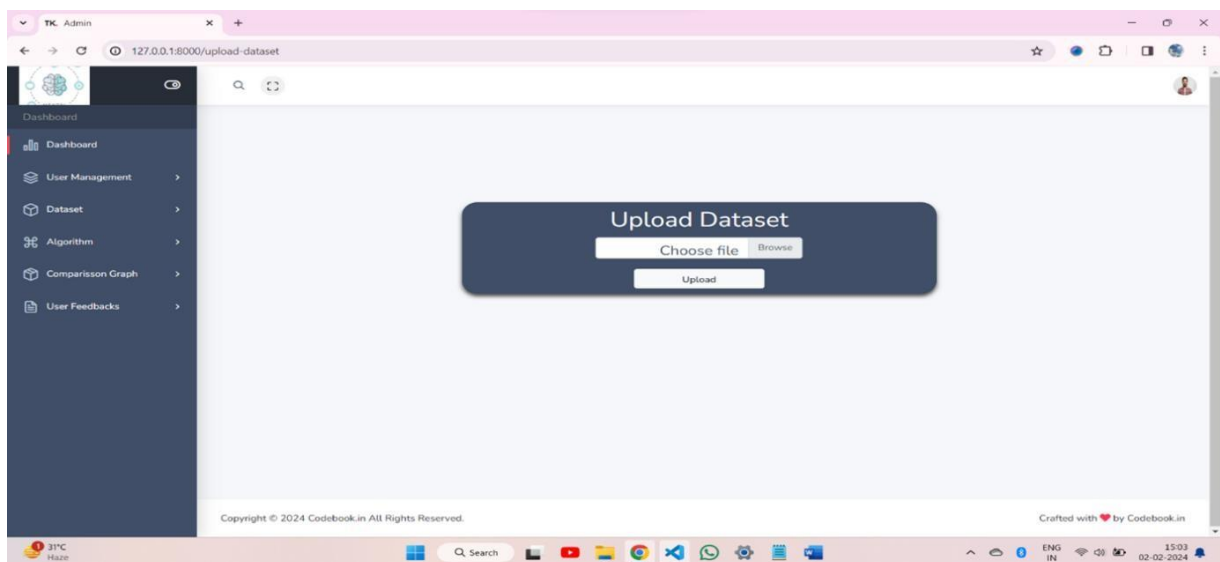


fig 17. Dataset upload

STEP-9:After running the algorithms successfully a comparison graph is formed.

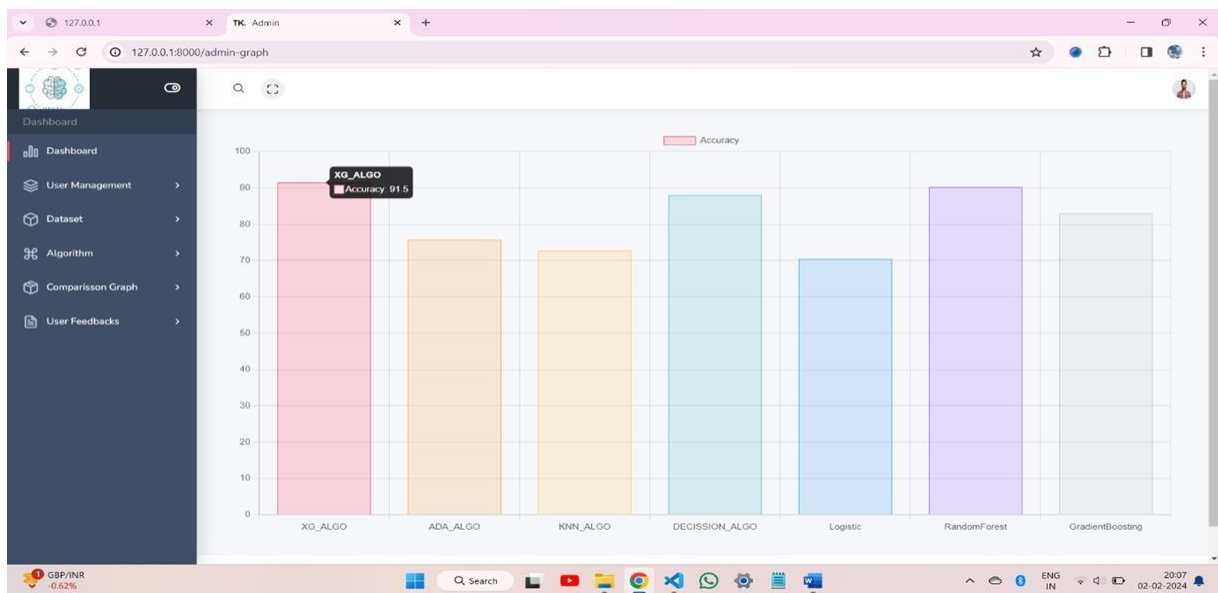


fig 18. Comparison graph

STEP-10: So, It would be easy for the user to know about their mental health condition by filling their respective details.

Mental Health Prediction

Age:

Physical Health Interview:

Care Options:

Medical Leave:

Benefits:

Employee Count Company:

Family History:

Work Interfere:

fig 19. Prediction form

STEP-11: If they don't have any issues it would show as "Mental Health Not Detected". Else it would show "Mental Health Detected".

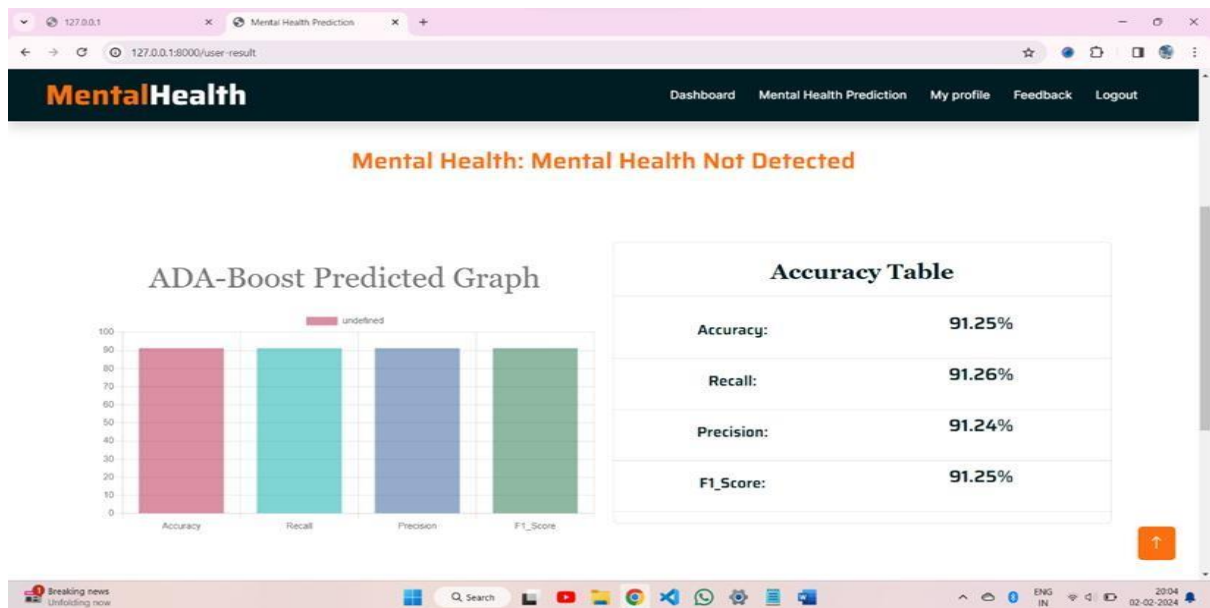


Fig 20. Not detected

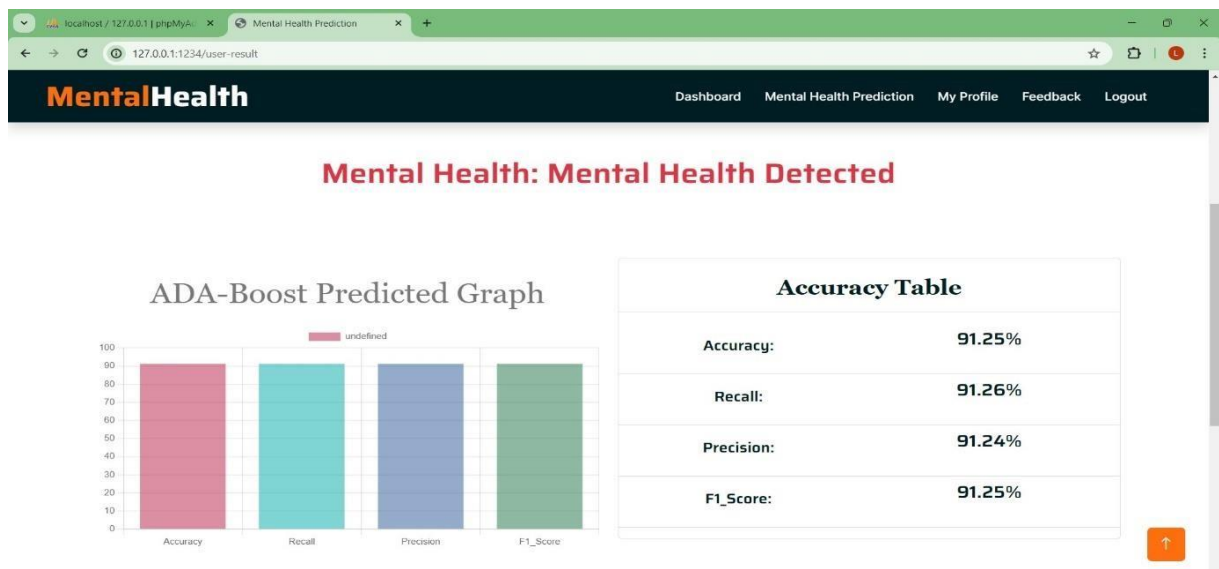


Fig 21. Detected graph

CHAPTER-8

CONCLUSION& FUTURE WORK

In the medical field, there are numerous specialised programmes that can accurately and timely forecast disease, allowing for effective and efficient treatment. Today, mental health is a major problem. As more people work from home and spend less time with their loved ones, the mental health problem has gotten worse. Therefore, it is crucial to spot any problems and deal with them before they have a significant impact. The development of a mental health illness tracker using machine learning algorithms has shown great potential for aiding in the identification and monitoring of mental health symptoms and behaviours.

The use of machine learning algorithms has allowed for the identification of patterns and trends in mental health data, which can help individuals, mental health professionals, and researchers to make more informed decisions and interventions. On successful implementation of Decision tree classifier, Random Forest, Naive Bayes, kNearest Neighbors, and Support Vector Machine algorithms, we have found Support Vector Machine algorithm gives the best results as it works for both the classification and regression task, although it was the most complex to implement. Support Classification typically employs vector machines. The exceptional classification performance and presentation quality of this classifier, which separates the data linearly into two unique classes (also known as hyperplanes) with the maximum distance between the two classes, have recently been used in numerous applications. After completing an online questionnaire on mental health conditions, a user will get to know whether they are suffering from stress levels, anxiety and depression. Furthermore, demographic questions on the participants' age, gender, and level of education will be asked.

In the realm of predicting mental health illness using machine learning, future endeavors hold the promise of significant advancements in both accuracy and applicability. One pivotal direction for research lies in the integration of multifaceted data streams. By incorporating not only traditional clinical data but also emerging sources like genetic markers, neuroimaging results, and even behavioral patterns gleaned from social media, predictive models can attain a more holistic understanding of an individual's mental health profile. Furthermore, longitudinal analyses present an avenue for capturing the dynamic nature of mental health conditions over time. By tracking individuals across intervals, predictive models can discern patterns of progression or remission, thereby enabling early interventions and personalized treatment plans.

REFERENCES

- [1] Vaishnavi, Konda, et al. "Predicting Mental Health Illness using Machine Learning Algorithms." Journal of Physics: Conference Series. Vol. 2161. No. 1. IOP Publishing, 2022.
- [2] R. J. S. A. R. J. C. L. Ariel S. Teles, "Towards Situationaware Mobile Application in Mental Health",2022.
- [3] Satvik G, Chetna P, Ritesh S, Mental Health Prediction Using Machine Learning, International Research Journal of Engineering and Technology (IRJET), Volume: 09 Issue:12, Dec 2022, Pune, India.
- [4] Priya, Anu & Garg, Shruti & Tigga, Neha. (2020). Predicting Anxiety, Depression and Stress in Modern Life using Machine Learning Algorithms,2020. Procedia Computer Science. 167. 1258-1267. 10.1016/j.procs.2020.03.442.
- [5] I. R. M. E. R. R. Ariel Teles, "Mobile Mental Health: A Review of Applications for Depression Assistance," in 2019 IEEE 32nd International Symposium on Computer-Based Medical System (CBMS), 2019.
- [6] V. M. Vidhi Mody, "Mental Health Monitoring System Using Artificial Intelligence: A Review," in 2019 5th International Conference for Convergence in Technology (I2CT), Pune, 2019.
- [7] S. G. Alonso, I. Torre-Díez, S. Hamrioui, M.l LopezCoronado, D. C. Barreno, L. M. ' Nozaleda, and M. Franco. Data Mining Algorithms and Techniques in Mental Health: A Systematic Review. J. Med. Syst. Vol. 42,2018.
- [8] U. S. Reddy, A. V. Thota and A. Dharun, "Machine Learning Techniques for Stress Prediction in Working Employees," 2018 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), Madurai, India, 2018.
- [9] Einstein, A., B. Podolsky, and N. Rosen, 1935, "Can quantummechanical description of physical reality be considered complete?", Phys. Rev. 47, 777-780.
- [10] M. P. Dooshima, E. N. Chidozie, B. J. Ademola, O. O. Sekoni, I. P. Adebayo, A Predictive Model for the Risk of Mental Illness in Nigeria Using Data Mining, International Journal of Immunology. Vol. 6, No. 1, 2018, pp. 5-16.

- [11] M. Srividya, M. Subramaniam and B. Natarajan, “Behavioral Modeling for Mental Health using Machine Learning Algorithms” “Journal of Medical Systems” Vol. 42(5):88 May 2018.
- [12] D.Filip & C. Jesus. (2015). A Neural Network Based Model for Predicting Psychological Conditions International Conference on Brain Informatics and Health 252-261.

APPENDIX-A

SOURCE CODE

```
from django.shortcuts import render, redirect
import urllib.parse #for send SMS
import urllib.request #for send SMS
import time
import random
from userapp.models import * # * will import everything
from django.conf import settings
from django.core.files.storage import default_storage
from django.conf import settings
from django.contrib import messages
from django.core.mail import send_mail
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd
import imblearn
from imblearn.over_sampling import SMOTE
from sklearn.metrics import r2_score
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

from django.contrib import messages
import pickle
from adminapp.models import *

# Create your views here.
```

```
def sendSMS(user, otp, mobile):
    data = urllib.parse.urlencode({
        'username': 'Codebook',
        'apikey': '56dbbdc9cea86b276f6c',
        'mobile': mobile,
        'message': f'Hello {user}, your OTP for account activation is {otp}. This message is
generated from https://www.codebook.in server. Thank you',
        'senderid': 'CODEBK'
    })
    data = data.encode('utf-8')
    # Disable SSL certificate verification
    # context = ssl._create_unverified_context()
    request = urllib.request.Request("https://smslogin.co/v3/api.php?")
    f = urllib.request.urlopen(request, data)
    return f.read()

def user_profile(request):
    views_id = request.session['user_id']
    user = UserDetails.objects.get(user_id = views_id)
    if request.method == 'POST':
        username = request.POST.get('full_name')
        email = request.POST.get('email_address')
        phone = request.POST.get('Phone_number')
        password = request.POST.get('pass')
        date = request.POST.get('date')
        print(username, email, phone, password, 'data')

        user.user_username = username
        user.user_email = email
        user.user_contact = phone
        user.user_password = password
```

```
if len(request.FILES)!= 0:
    image = request.FILES['image']
    user.user_image = image
    user.user_username = username
    user.user_email = email
    user.user_contact = phone
    user.user_password = password
    user.save()
    messages.success(request, 'Updated Successfully...!')

else:
    user.user_username = username
    user.user_email = email
    user.user_contact = phone
    user.user_password = password
    user.save()
    messages.success(request, 'Updated Successfully...!')

return render(request,'user/profile.html', {'i':user})

def user_about(request):
    return render(request,'user/about.html')

def user_admin(request):
    admin_email= "admin@gmail.com"
    admin_password="admin"
    if request.method=="POST":
        admin_e=request.POST.get("admin_email")
        admin_p=request.POST.get("admin_password")
        if (admin_e==admin_email and admin_p==admin_password):
            messages.success(request,'login successfull')
            return redirect("admin_dashboard")
```

```
else:
    messages.error(request,"login credentials was incorrect. ..")
    return redirect("user_admin")
return render(request,'user/admin.html')

def user_contact(request):
    return render(request,'user/contact.html')

def user_dashboard(request):
    prediction_count = UserDetails.objects.all().count()
    user_id = request.session["user_id"]
    user = UserDetails.objects.get(user_id = user_id)
    return render(request,'user/user-dashboard.html', {'predictions': prediction_count, 'la': user})

def user_feedback(request):
    views_id = request.session['user_id']
    user = UserDetails.objects.get(user_id = views_id)
    if request.method == 'POST':
        u_feedback = request.POST.get('feedback')
        u_rating = request.POST.get('rating')
        if not user_feedback:
            return redirect("")
        sid=SentimentIntensityAnalyzer()
        score=sid.polarity_scores(u_feedback)
        sentiment=None
        if score['compound']>0 and score['compound']<=0.5:
            sentiment='positive'
        elif score['compound']>=0.5:
            sentiment='very positive'
        elif score['compound']<-0.5:
            sentiment='very negative'
```

```
elif score['compound']<0 and score['compound']>=-0.5:
    sentiment='negative'
else :
    sentiment='neutral'
messages.success(request,'Feedback sent successfully')

print(sentiment)
user.star_feedback=u_feedback
user.star_rating = u_rating
user.save()

UserFeedbackModels.objects.create(user_details = user, star_feedback = u_feedback,
star_rating = u_rating, sentment= sentiment)
rev=UserFeedbackModels.objects.filter()

return render(request,'user/user-feedback.html')

def user_forgotpassword(request):
    return render(request,'user/user-forgotpwd.html')

def user_index(request):
    return render(request,'user/user-index.html')

def user_login(request):
    if request.method == 'POST':
        email = request.POST.get('user_email')
        password = request.POST.get('user_password')
        print(email, password,'data')
        try:
            user = UserDetails.objects.get(user_email = email, user_password = password)
            print(user)

            if user.user_password == password :
```

```
if user.user_status == 'Accepted':
    if user.otp_status == 'verified':
        messages.success(request,'login successfull')
        request.session['user_id'] = user.user_id
        print('login sucessfull')
        user.no_of_times_login += 1
        user.save()
        return redirect('user_dashboard')
    else:
        return redirect('user_otp')
elif user.user_password == password and user.user_status == 'Rejected':
    messages.warning(request,"you account is rejected")
else:
    messages.info(request,"your account is in pending")
else:
    messages.error(request,'Login credentials was incorrect...')
except:
    messages.error(request,'Login credentials was incorrect...')
    print(';invalid credentials')
    print('exce ')
    return redirect('user_login')

return render(request,'user/user-login.html')
```

```
def user_prediction(request):
    if request.method == 'POST':
        Physical_Health_Interview = request.POST.get('Physical_Health_Interview')
        Care_Options = request.POST.get('Care_Options')
        Medical_Leave = request.POST.get('Medical_Leave')
        Benefits = request.POST.get('Benefits')
        Employee_Count_Company = request.POST.get('Employee_Count_Company')
        Family_History = request.POST.get('Family_History')
```

```
Age = request.POST.get('Age')

Work_Interfere = request.POST.get('Work_Interfere')
print(Physical_Health_Interview,Care_Options,Medical_Leave,Benefits,Employee_Count_C
ompany,Family_History,Age,Work_Interfere,'data')

Physical_Health_Interview = int(Physical_Health_Interview)

Care_Options = int(Care_Options)

Medical_Leave = int(Medical_Leave)

Benefits = int(Benefits)

Family_History = int(Family_History)

Age = int(Age)

Employee_Count_Company = int(Employee_Count_Company)

Work_Interfere = int(Work_Interfere)

file_path = 'mental_health_dataset/random.pkl'

with open(file_path, 'rb') as file:

    loaded_model = pickle.load(file)

# res = loaded_model.predict([[ 2,0,2,1,3,1,26,3]])

#print(res,'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa')

res = loaded_model.predict([[ Physical_Health_Interview,
Medical_Leave,Care_Options,Benefits,Employee_Count_Company,Family_History,Age,Wo
rk_Interfere]])

dataset = Upload_dataset_model.objects.last()

df=pd.read_csv(dataset.Dataset.path)

X = df.drop('Treatment', axis = 1)

y = df['Treatment']

res_list = res.tolist()

request.session['result'] = res_list

print(res_list,'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa')

from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,random_state=1,test_size=0.2)
from sklearn.ensemble import HistGradientBoostingClassifier
GB = HistGradientBoostingClassifier()
GB.fit(X_train, y_train)

# # prediction
train_prediction= GB.predict(X_train)
test_prediction= GB.predict(X_test)
# print('*'*20)

accuracy = round(accuracy_score(y_test,test_prediction)*100, 2)
print(accuracy,"hgfcyhtgrdthr dthdtrhdtrhdtrdtrdtrdtr")
precession = round(precision_score(y_test,test_prediction,average = 'macro')*100, 2)
recall = round(recall_score(y_test,test_prediction,average = 'macro')*100,2)
f1 = round(f1_score(y_test,test_prediction,average = 'macro')*100, 2)
request.session['acr'] = accuracy
request.session['pre'] = precession
request.session['rec'] = recall
request.session['f'] = f1
print(res_list, "result")
return redirect('user_result')
return render(request,'user/prediction.html')

def user_result(request):
    messages.success(request,"Social Crime Has Been Detected")
    accuracy = request.session.get('acr')
    precession = request.session.get('pre')
    recall = request.session.get('rec')
    f1 = request.session.get('f')
    res = request.session.get('result')
```



```
print(accuracy,"tttttttttttttttttttttt")
```

```
print(res,"xaxfdgdyfur")
```

```
return render(request,'user/result.html',{'accuracy':accuracy, 'precession':precession,
'recall': recall, 'f1':f1, 'result':res})
```

```
def user_otp(request):
    user_id=request.session["user_email"]
    user=UserDetails.objects.get(user_email=user_id)
    print(user_id)
    print(user,"user")
    print(user.otp,"create_otp")
    if request.method=="POST":
        u_otp=request.POST.get("user_otp")
        u_otp=int(u_otp)
        print(u_otp,"otp")
        if u_otp==user.otp:
            print("if")
            user.otp_status="verified"
            user.save()
            return redirect ("user_login")
        else:
            print("else")
            return redirect("user_otp")

    return render(request,'user/otp.html')
```

```
def user_register(req):
    if req.method == "POST":
        username=req.POST.get("register_username")
        email=req.POST.get("register_email")
```

```
password=req.POST.get("register_password")
contact=req.POST.get("register_contact")
profile=req.FILES["register_choosefile"]
print(req)
print(username,email,password,contact,profile,"register")
otp=str(random.randint(1000,9999))
print(otp, "generate otp")
try:
    user_data = UserDetails.objects.get(user_email=email)
    return redirect("user_register")
except:
    mail_message=f"register successfully \nYour four digit pin is below \n {otp}"
    print(mail_message)
    send_mail("student password",mail_message,settings.EMAIL_HOST_USER,
[email]) #we take the register name variable in this case which is "email"
    #send SMS
    sendSMS(username, otp, contact)

    UserDetails.objects.create(otp=otp, user_username=username, user_email=email,
user_password=password, user_contact=contact,user_image=profile)
    req.session["user_email"] = email
    return redirect ("user_otp")

return render(req,'user/register.html')

def userlogout(request):
    view_id = request.session["user_id"]
    user = UserDetails.objects.get(user_id = view_id)
    t = time.localtime()
    user.last_login_time = t
    current_time = time.strftime('%H:%M:%S', t)
    user.last_login_time = current_time
    current_date = time.strftime('%Y-%m-%d')
```

```
user.last_login_date = current_date
user.save()
messages.info(request, 'You are logged out..')
# print(user.Last_Login_Time)
# print(user.Last_Login_Date)
return redirect('user_login')
```

PAPER PUBLICATION



UTILIZING MACHINE LEARNING TO ENHANCE EARLY INTERVENTION IN MENTAL HEALTH CARE

Dr. M.Uma Devi¹, Nurukurthy L V V Aditya², Y Lakshmi Prasanna³, A Prame Viswanadh⁴, P Lakshmi Priyanka⁵

¹Assistant Professor , Department of Information Technology, Pragati Engineering College, Surampalem , Andhra Pradesh, India

^{2, 3, 4, 5}B.tech Students Department of Information Technology, Pragati Engineering College, Surampalem , Andhra Pradesh, India

Abstract: People these days are increasingly concerned with their physical health, but they are paying less attention to their mental health. Many people refuse to seek therapy despite being aware that they have chronic mental illnesses. This can be due to a multitude of factors, including a dislike for doctors, a fear of what others will think, or the belief that they have gone insane. Given these circumstances, a solution must be developed immediately to avoid the spread of mental diseases to more people. The primary purpose of this study is to predict mental health using support vector machines and deep learning approaches. As many machine learning and deep learning techniques help to overcome these modern challenges, support vector machines are employed to handle the current problem. Support Vector Machines (SVM) outperform other machine learning algorithms in terms of mental disease prediction.

Index Terms -Machine learning, Mental Illness, Support Vector Machine, Decision Tree.

I. INTRODUCTION

Mental illness is a medical condition that clearly impacts a person's emotions, intelligence, and capacity to communicate with others. These issues have highlighted the tremendous impact of mental illness on society, as well as the need for new treatment and preventative techniques. Prior to implementing these procedures, early mental health screening must be undertaken. Because mental disease diagnosis is frequently based on the patient's self-report, it is vital to use questionnaires designed to uncover specific emotional or social interaction patterns. Many persons with emotional disorders or mental illnesses should be able to recover with the proper care and therapy. Machine learning is a widely used subfield of artificial intelligence today. It is thought to be a very good tool for mental health prediction. It enables many researchers to gain important insights from data, deliver personalized experiences, and create automated intelligent systems. Support vector machines, random forests, and artificial neural networks are some of the most common machine learning approaches used to predict and categorize future events. Machine learning (ML) is improving its ability to identify diseases while also providing clinicians with a platform to study massive volumes of patient data and give personalized treatments based on the patient's medical state. People have grown accustomed to working from home as a result of the advancement of new technology, and many software and IT firms have embraced this concept. People rarely connect with one another as a result, and many lead unhealthy lives at home. As a result, many people today endure feelings of depression, anxiety, and hopelessness. In today's world, people place a higher value on physical health than mental health. Many people choose not to seek therapy, even if they are aware that they have long-term mental health issues, because they dislike doctors, are frightened of being judged, or believe they are insane [1]. The primary goal of this strategy is to develop a system capable of reliably and accurately preserving original work while also anticipating users' mental health concerns.

II. LITERATURE SURVEY

The study [1], A Mental Health Tracker Built Using Flutter and Firebase, focuses on daily questions from the application that ask the user about recent events and their emotions. It uses natural language processing and a lexicon-based technique to automatically determine a user's core emotion from natural language input. The application provides a suitable measurement based on the emotion, such as activities or mindfulness exercises. The work [2] Predicting Mental Health Illness Using Machine Learning Algorithms focuses on five machine learning methods: random forest, decision tree stacking, logistic regression, and k nearest neighbour classifier. They also assessed how effectively they could identify mental health issues. More than 79% of classifiers have an accuracy rate. Although the study employed very little data, it may still be useful for increased accuracy if a larger data set is used in the future. This paper, Mental Health Prediction Using Machine Learning: Taxonomy, Applications, and Challenges (paper[3]), reviews and evaluates thirty research papers that emphasize the application of machine learning techniques or approaches in predicting mental health disorders. The research papers and articles are divided and classed based on the numerous mental health issues, such as bipolar disorder, schizophrenia, depression, anxiety, and PTSD. The paper[4], Mental Health Prediction using Machine Learning: A Systematic Review, is based on an open dataset obtained through an OSMI (Open Sourcing Mental Illness) survey. The vast majority of the data in the collection pertains to employed individuals. The majority of the data consists of Yes/No responses indicating whether or not a person should seek therapy. The paper[5] describes a mental health tracker that asks the user a number of questions. Based on the user's response, machine learning techniques such as random forest and natural language processing (NLP) will



be utilized to determine the user's current mental state and mood. The Depression, Anxiety, and Stress Scale questionnaire was used to collect data from employed and unemployed persons from various cultures and groups for the publication [6], Predicting Stress, Depression, and Anxiety in Modern Life. Predicting medical personnel's mental health during COVID-19 is a paper[7] that focuses on one of the most important aspects of lowering the risk of major mental illness. It predicts the mental health of healthcare practitioners using 32 factors.

III. SYSTEM ANALYSIS

A. EXISTING SYSTEM

Of course, here's a ten-line summary of the existing system:

The current mental health diagnosis is primarily based on subjective evaluations given by medical practitioners.

Making a diagnosis takes time since it sometimes necessitates comprehensive surveys and interviews.

The stigma associated to mental health problems undermines the system by discouraging people from seeking treatment.

It is further limited by unequal access to care and a scarcity of mental health professionals.

Traditional diagnostic procedures cannot be scaled, limiting early intervention.

Data collection for mental health analysis can be erratic, non-standardized, and insufficient.

Different practitioners may diagnose patients with varying levels of precision and accuracy.

The present technological tools for identifying mental health concerns have limitations and may not be widely adopted.

It is likely that patients' perspectives on mental health care are not given adequate consideration.

These difficulties highlight the importance of machine learning and other more effective, data-driven strategies for improving mental health diagnosis.

DISADVANTAGES OF THE EXISTING SYSTEM

- 1. Subjectivity:** One fundamental disadvantage of existing diagnostic approaches is their reliance on medical professionals' subjective opinions, which can lead to inconsistent and variable diagnoses.
- 2. Stigma:** The stigma associated with mental health issues often discourages people from seeking help, resulting in underreporting and delayed treatment.
- 3. Scalability:** Because traditional diagnosis methods are time-consuming, they may fail to meet the expanding demand for mental health therapies.
- 4. Access and Availability:** If mental health professionals are not equitably accessible, treatment may be delayed or inadequate, particularly in remote areas.
- 5. Data Collection Challenges:** The existing system's irregular, insufficient, and non-standardized data collection makes it difficult to generate comprehensive mental health profiles.
- 6. Accuracy and Consistency:** Practitioners may diagnose patients differently or inconsistently, resulting in delayed or inaccurate diagnosis.
- 7. Limited technology integration:** Because of their limitations, the healthcare system may not always adopt the new technology tools available for detecting mental health concerns.
- 8. Financial Barriers:** For many people, the cost of mental health services like counseling and medication can be a significant impediment to seeking treatment.
- Wait Times:** Long wait times for mental health appointments can increase patients' symptoms and postpone treatment.
- 9. Lack of Patient-Centric Care:** The existing system may not adequately include patients' perspectives and preferences when developing mental health treatment programs.

These limitations underscore the need for more effective, patient-centered, data-driven technologies, such as machine learning, to improve mental health diagnosis and treatment. Some of these issues can be addressed through your research by providing a more exact and expandable replacement.

B. PROPOSED SYSTEM

The proposed strategy, which employs cutting-edge machine learning techniques, aims to address the limitations of the present mental health diagnosis paradigm, with a primary focus on Support Vector Machine (SVM). This unique approach to mental health assessment will include an objective, data-driven component. By assessing a variety of patient data, such as behavioral patterns, physiological markers, and psychological evaluations, the proposed approach will allow for more accurate and consistent diagnoses. Furthermore, it will provide a stigma-free and judgment-free environment, encouraging people to seek help and play an active part in managing their mental health. The system's scalability will ensure timely and



convenient access to mental health care, addressing the current shortage of mental health specialists in certain places. By integrating technological advances and data-driven insights, the proposed strategy has the potential to alter mental health diagnosis and provide tailored, patient-centred therapy. This will significantly enhance the overall well-being of persons battling with mental health issues.

IV.

SYSTEM DESIGN

SYSTEM ARCHITECTURE

Below diagram depicts the whole system architecture.

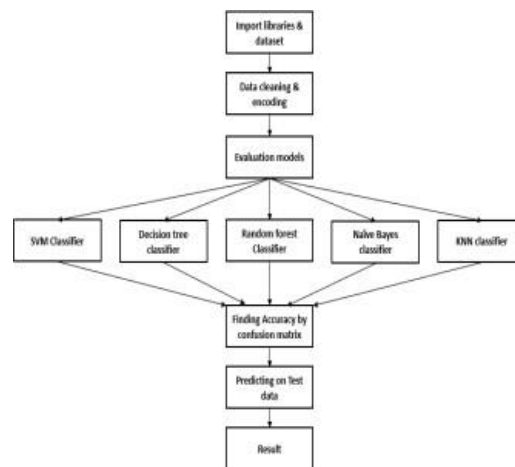


Fig 1. Methodology followed for proposed model

V. SYSTEM IMPLEMENTATION

MODULES

1. Data Collection Module: This module is responsible for gathering data from various sources, such as psychological assessments, biometric sensors, and behavioral data. It ensures that data is securely and efficiently collected and stored for analysis.

2. Data Preprocessing Module: Before analysis, raw data is often noisy and requires cleaning, transformation, and normalization. This module preprocesses the data to ensure it is in a suitable format for machine learning algorithms.

3. Feature Selection and Engineering Module: Feature selection is essential for identifying the most relevant data attributes that contribute to mental health prediction. Feature engineering may involve creating new attributes or aggregating existing ones for improved prediction.

4. Machine Learning Algorithm Module: The system's foundational module uses machine learning methods, specifically Support Vector Machine (SVM), to predict mental health.

It trains the models on preprocessed data and then uses them to produce batch or real-time predictions.

5. User Interface Module: An easy-to-use interface allows users to communicate with the system, provide data, and receive ideas or forecasts.

It may also include visualization features to help consumers better comprehend their mental health status.

6. Database and Data Management Module: This module manages how data is stored and retrieved while ensuring data security, privacy, and compliance with applicable regulations.

7. Report Generation and Analysis Module: This module assists users and medical specialists in understanding the results by providing reports and summaries of the predictions.

This module may also include analytical tools for conducting detailed data analysis.

8. Security and Privacy Module: When it comes to mental health, data security and privacy are essential. This module ensures that the access control, encryption, and privacy rules are obeyed.

9. Integration with Healthcare Systems Module: If applicable, this module facilitates the system's integration with existing healthcare systems, allowing for seamless data sharing and communications.

10. Feedback and Improvement Module: Using feedback methods, the system can evolve and alter over time, boosting



user satisfaction and forecast accuracy.

11. Training and assistance Module: Provides training and assistance to users of the system, including healthcare professionals, so that they can fully utilize its capabilities.

12. Study and Development Module: By providing anonymized and aggregated data to academics in the field, this module facilitates ongoing research in mental health prediction.

Together, these modules comprise the proposed machine learning-based mental health prediction system. Every module contributes to the system's efficiency, safety, and usability. You can go into greater detail about each module in your thesis, describing its function, the algorithms it employs, and how it aids the system in achieving its objectives.

VI. RESULTS AND DISCUSSION

The outcomes range from 0 to 1. SVM is quite exact. A graph depicts the general behavior of algorithms when a dataset is used to get the intended result. At 1.00, the SVM method is the most accurate, followed by Nave Bayes. Random Forest and Naive Bayes yield the highest precision values. The best recall and F1 score go to SVM. The findings of these algorithms reveal that SVM has the highest level of accuracy for attaining the desired and correct outputs.

Figure 1: Login Page

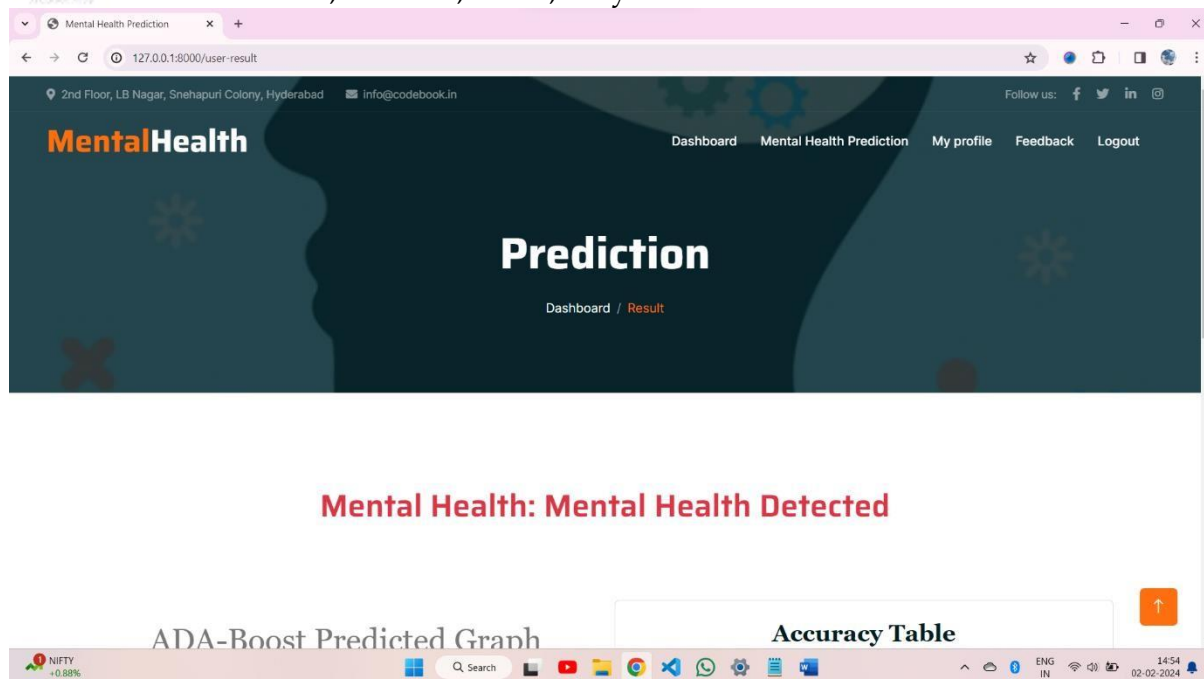


Figure 2: Mental health detected

VII. CONCLUSION AND FUTURE WORK

Many specialized medical systems are available that can accurately and quickly forecast disease, allowing for effective and efficient treatment. Mental health issues are a major concern today. The problem of mental health has worsened as more people work from home and spend less time with their families and friends. As a result, it is critical to detect and resolve any difficulties as quickly as possible to avoid more catastrophic repercussions. The use of machine learning algorithms to create a tracker for mental health diseases has shown great potential in assisting with the recognition and observation of mental health behaviours and symptoms.

The use of machine learning algorithms has made it feasible to detect patterns and trends in mental health data, allowing individuals, mental health professionals, and researchers to make better decisions and apply therapies. After successfully implementing the Decision Tree classifier, Random Forest, Naive Bayes, kNearest Neighbors, and Support Vector Machine algorithms, we discovered that, while the Support Vector Machine algorithm is the most difficult to implement, it produces the best results because it can be used for both classification and regression. Vector machines are widely used for support categorization. This classifier separates the data linearly into two unique classes (also known as hyperplanes) based on the biggest distance between them. Its outstanding classification performance and presentation quality have recently led to its use in a wide range of applications. After taking an online mental health quiz, a user will learn whether they are feeling stress, anxiety, or depression. In addition, demographic questions on the participants' age, gender, and educational achievement will be asked. The information gathered will remain anonymous and confidential.

REFERENCES :

- [1] Vaishnavi, Konda, et al. "Predicting Mental Health Illness using Machine Learning Algorithms." Journal of Physics: Conference Series. Vol. 2161. No. 1. IOP Publishing, 2022.
- [2] R. J. S. A. R. J. C. L. Ariel S. Teles, "Towards Situationaware Mobile Application in Mental Health", 2022.
- [3] Satvik G, Chetna P, Ritesh S, Mental Health Prediction Using Machine Learning, International Research Journal of Engineering and Technology (IRJET), Volume: 09 Issue:12, Dec 2022, Pune, India.
- [4] Priya, Anu & Garg, Shruti & Tigga, Neha. (2020). Predicting Anxiety, Depression and Stress in Modern Life using Machine Learning Algorithms, 2020. Procedia Computer Science. 167. 1258-1267. 10.1016/j.procs.2020.03.442.
- [5] I. R. M. E. R. R. Ariel Teles, "Mobile Mental Health: A Review of Applications for Depression Assistance," in 2019 IEEE 32nd International Symposium on Computer-Based Medical System (CBMS), 2019.
- [6] V. M. Vidhi Mody, "Mental Health Monitoring System Using Artificial Intelligence: A Review," in 2019 5th International Conference for Convergence in Technology (I2CT), Pune, 2019.
- [7] S. G. Alonso, I. Torre-Díez, S. Hamrioui, M. I. Lopez Coronado, D. C. Barreno, L. M. Nozalea, and M. Franco. Data



Mining Algorithms and Techniques in Mental Health: A Systematic Review. J. Med. Syst. Vol. 42,2018.

[8] U. S. Reddy, A. V. Thota and A. Dharun, "Machine Learning Techniques for Stress Prediction in Working Employees," 2018 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC), Madurai, India, 2018.

[9] Einstein, A., B. Podolsky, and N. Rosen, 1935, "Can quantummechanical description of physical reality be considered complete?", Phys. Rev. 47, 777-780.

[10] M. P. Dooshima, E. N. Chidozie, B. J. Ademola, O. O. Sekoni, I. P. Adebayo, A Predictive Model for the Risk of Mental Illness in Nigeria Using Data Mining, International Journal of Immunology. Vol. 6, No. 1, 2018, pp. 5-16.

[11] M. Srividya, M. Subramaniam and B. Natarajan, "Behavioral Modeling for Mental Health using Machine Learning Algorithms" "Journal of Medical Systems" Vol. 42(5):88 May 2018.

[12] D.Filip & C. Jesus. (2015). A Neural Network Based Model for Predicting Psychological Conditions International Conference on Brain Informatics and Health 252-261.